```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using Sy.G11.Game;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.G11.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S11_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot             UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            // The single-mode switch destroys S00_Boot (and this coroutine's host with
            // it) before the wait below could ever resume, so WireMainHost rode a frame
            // that never came and the main scene stayed a shell (X1909 live probe).
            // The completion event is a plain delegate: it survives the switch and
            // runs after the new scene is active; WireMainHost's own null-guard keeps
            // the wiring idempotent.
            if (op != null) op.completed += _ =&gt; WireMainHost();
            yield break;
        }
        /// &lt;summary&gt;T005: the main scene file stays a shell; the wage HUD host is wired at runtime.&lt;/summary&g
        static void WireMainHost()
```

```
        {
            if (FindObjectOfType&lt;WorkerMainHost&gt;() != null) return;
            var hostGo = new GameObject("WorkerMainHost");
            hostGo.AddComponent&lt;WorkerMainHost&gt;();
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.G11.Boot
{
    /// &lt;summary&gt;
    /// Boot     T001
    /// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
    ///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
    /// X006      SY_MOCK
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
```

```
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; //              JS sink platform T001c
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G11.Boot
{
    /// &lt;summary&gt;
```

```
///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
/// T001c  WX/TT     Channel   Editor      Boot
/// &lt;/summary&gt;
public static class BootServices
{
    public static void RegisterAll()
    {
        if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
        // T009 slice 1b: local telemetry with the Editor-domain debug sink
        // (P02 boot paradigm) - the offline funnel trail rides evt_ad now;
        // the Wx/TT analytics sink swaps in with the real suite (T001c).
        var telemetry = new Sy.Telemetry.TelemetryService();
        telemetry.AddSink(new Sy.Telemetry.DebugLogSink());
        ServiceLocator.Register(telemetry);
        Logger.Info("BOOT", "mock services registered (SY_MOCK)");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                   TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
```

```
        public SyncResult CloudSync() =&gt; new SyncResult(false, null);
    }
    public sealed class MockLocaleService : ILocaleService
    {
        public void SetLang(string lang) { }
        /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        public FontSet GetFontSet()
            =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
    }
    public sealed class MockAdsService : IAdsService
    {
        public event Action&lt;AdLifecycle&gt; OnAdEvent;
        public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
            =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
        public void ShowInterstitial(string pointId) { }
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// WorkerAboutPanel.cs
using System;
using System.Globalization;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T020 compliance surface - the about/disclosure face on the main menu
    /// (platform AIGC labeling rule + random-content disclosure, X420 items
    /// five and six). Every disclosure line is composed FROM the live law
    /// tables (gacha pool tiers, AdCapPolicy category caps) so the published
    /// numbers can never drift from the enforced ones - the same construction
    /// guarantee the P02/P06 about panels use. The composer layer is pure
    /// static; the headless gate asserts it without rendering. Programmatic
    /// UGUI in the electro palette, zero scene-YAML drift, presentation-only.
    /// Display text rides the locale-ready lookup (zh table when loaded,
    /// English source otherwise) - the same contract every WorkerTexts face
    /// ships with.
    /// &lt;/summary&gt;
    public sealed class WorkerAboutPanel : MonoBehaviour
    {
```

```
public const string KeyAboutButton = "about_button";
public const string KeyPanelTitle = "about_title";
public const string KeyAigcLine = "about_aigc_line";
public const string KeyOddsLine = "about_odds_line";
public const string KeyAdCapLine = "about_adcap_line";
public const string KeyAgeLine = "about_age_line";
public const string KeyCloseLabel = "about_close";
static readonly Color32 PanelBack = new Color32(0x1E, 0x22, 0x2C, 0xF2);
static readonly Color32 DimBack = new Color32(0x0D, 0x0F, 0x16, 0xC4);
static readonly Color32 SeatBack = new Color32(0x2A, 0x2E, 0x3A, 0xE6);
static readonly Color32 TextLight = new Color32(0xF2, 0xF4, 0xF8, 0xFF);
static readonly Color32 TextDim = new Color32(0x9A, 0xA2, 0xB4, 0xFF);
// ——— composer layer (pure static, gate-facing) ———
/// &lt;summary&gt;AIGC explicit disclosure: art AND audio assets in this
/// game are AI-assisted (T017 closed - the G11-A1 batch shipped the
/// seven AI clips; the platform-side declaration stays an admin
/// item). Downgrade law: should the audio face ever return to
/// procedural synthesis, this copy must drop the audio half the
/// same batch - two versions never ship together.&lt;/summary&gt;
public static string AigcLine()
{
    return Text(KeyAigcLine);
}
/// &lt;summary&gt;Ten-pull odds disclosure. The published numbers are
/// formatted straight out of the frozen pool's per-tier weight
/// sums, so the shown face and the enforced draw law are the same
/// object - a hot-tuned pool re-renders the disclosure with zero
/// extra wiring.&lt;/summary&gt;
public static string OddsLine()
{
    int common = 0, rare = 0, epic = 0, total = 0;
    foreach (var c in WorkerGacha.Pool)
    {
        total += c.Weight;
        if (c.Tier == WorkerGacha.Tier.Common) common += c.Weight;
        else if (c.Tier == WorkerGacha.Tier.Rare) rare += c.Weight;
        else epic += c.Weight;
    }
    return string.Format(CultureInfo.InvariantCulture, Text(KeyOddsLine),
        Percent(common, total), Percent(rare, total), Percent(epic, total));
}
/// &lt;summary&gt;Ad-cap disclosure: the four category ceilings formatted
/// out of the live AdCapPolicy tables (ten-pull / skip / boost /
/// offline-double), voluntary views only - no forced ad.&lt;/summary&gt;
public static string AdCapLine()
{
    return string.Format(CultureInfo.InvariantCulture, Text(KeyAdCapLine),
        CapOfPoint("g11_gacha_pull"), CapOfPoint("g11_event_skip"),
        CapOfPoint("g11_boost_30m"), CapOfPoint("g11_offline_double"));
}
```

```
/// &lt;summary&gt;Age-rating disclosure (IAA casual default: 8+; the
/// platform backend filing stays an admin item).&lt;/summary&gt;
public static string AgeLine()
{
    return Text(KeyAgeLine);
}
static int CapOfPoint(string pointId)
{
    return Sy.Platform.AdCapPolicy.CategoryCap(
        Sy.Platform.AdCategories.CategoryOfPoint(pointId));
}
static int Percent(int weight, int total)
{
    return total &gt; 0 ? (int)Math.Round(weight * 100.0 / total) : 0;
}
static string Text(string key)
{
    return WorkerTexts.LookupWithZh(WorkerTexts.LangZh, key);
}
// ——— view layer (programmatic UGUI, zero scene YAML) ———
RectTransform _overlay;
/// &lt;summary&gt;Builds the overlay under the HUD root, hidden until
/// Open. Pure view: reads only the composer layer above.&lt;/summary&gt;
public static WorkerAboutPanel AttachTo(Transform root)
{
    var go = new GameObject("AboutPanel", typeof(RectTransform));
    go.transform.SetParent(root, false);
    var panel = go.AddComponent&lt;WorkerAboutPanel&gt;();
    panel.Build();
    return panel;
}
void Build()
{
    _overlay = (RectTransform)transform;
    _overlay.sizeDelta = new Vector2(1080f, 1920f);
    var dim = NewImage(transform, "Dim");
    dim.color = DimBack;
    Stretch(dim.rectTransform);
    var back = NewImage(transform, "PanelBack");
    back.color = PanelBack;
    back.rectTransform.anchoredPosition = Vector2.zero;
    back.rectTransform.sizeDelta = new Vector2(960f, 1350f);
    var title = NewText(transform, "Title", Text(KeyPanelTitle), 51, WorkerPlaceholder.AccentElectro);
    title.rectTransform.anchoredPosition = new Vector2(0f, 570f);
    title.rectTransform.sizeDelta = new Vector2(780f, 105f);
    var aigc = NewText(transform, "AigcLine", AigcLine(), 36, TextLight);
    aigc.horizontalOverflow = HorizontalWrapMode.Wrap;
    aigc.rectTransform.anchoredPosition = new Vector2(0f, 375f);
    aigc.rectTransform.sizeDelta = new Vector2(840f, 150f);
    var odds = NewText(transform, "OddsLine", OddsLine(), 30, TextLight);
```

```
        odds.horizontalOverflow = HorizontalWrapMode.Wrap;
        odds.rectTransform.anchoredPosition = new Vector2(0f, 105f);
        odds.rectTransform.sizeDelta = new Vector2(840f, 330f);
        var adcap = NewText(transform, "AdCapLine", AdCapLine(), 30, TextDim);
        adcap.horizontalOverflow = HorizontalWrapMode.Wrap;
        adcap.rectTransform.anchoredPosition = new Vector2(0f, -225f);
        adcap.rectTransform.sizeDelta = new Vector2(840f, 195f);
        var age = NewText(transform, "AgeLine", AgeLine(), 33, TextDim);
        age.horizontalOverflow = HorizontalWrapMode.Wrap;
        age.rectTransform.anchoredPosition = new Vector2(0f, -435f);
        age.rectTransform.sizeDelta = new Vector2(840f, 105f);
        var closeGo = new GameObject("CloseBtn", typeof(RectTransform), typeof(Image), typeof(Button));
        closeGo.transform.SetParent(transform, false);
        var closeRt = (RectTransform)closeGo.transform;
        closeRt.anchoredPosition = new Vector2(0f, -600f);
        closeRt.sizeDelta = new Vector2(630f, 135f);
        var closeImg = closeGo.GetComponent&lt;Image&gt;();
        closeImg.color = SeatBack;
        var closeLabel = NewText(closeGo.transform, "Label", Text(KeyCloseLabel), 39, TextLight);
        closeLabel.rectTransform.sizeDelta = new Vector2(600f, 120f);
        closeLabel.raycastTarget = false;
        closeGo.GetComponent&lt;Button&gt;().onClick.AddListener(Close);
        _overlay.gameObject.SetActive(false);
    }
    public void Open()
    {
        if (_overlay != null) _overlay.gameObject.SetActive(true);
    }
    // T024-L3 P2 (X1428): public so the host's single-modal stack law
    // can retire this overlay when another modal opens.
    public void Close()
    {
        if (_overlay != null) _overlay.gameObject.SetActive(false);
    }
    // ——— UGUI helpers (host-local style, no shared dependencies) ———
    static Text NewText(Transform parent, string name, string str, int size, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform));
        go.transform.SetParent(parent, false);
        var text = go.AddComponent&lt;Text&gt;();
        // T024-L2a2: this panel IS the zh face (every line composes through
        // LookupWithZh), so it must ride the shipped CJK subset - the
        // built-in carries zero zh glyphs and tofus on real packages.
        text.font = WorkerFonts.Resolve(true);
        text.text = str;
        text.fontSize = size;
        text.color = color;
        text.alignment = TextAnchor.MiddleCenter;
        text.horizontalOverflow = HorizontalWrapMode.Overflow;
        text.verticalOverflow = VerticalWrapMode.Overflow;
```

```
            return text;
        }
        static Image NewImage(Transform parent, string name)
        {
            var go = new GameObject(name, typeof(RectTransform));
            go.transform.SetParent(parent, false);
            return go.AddComponent&lt;Image&gt;();
        }
        static void Stretch(RectTransform rt)
        {
            rt.anchorMin = Vector2.zero;
            rt.anchorMax = Vector2.one;
            rt.offsetMin = Vector2.zero;
            rt.offsetMax = Vector2.zero;
        }
    }
}
// WorkerAudioCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// Real-clip audio catalog (T017, X027 two-state paradigm): the
    /// WorkerAudioSetup editor tool scans Assets/_Game/Art/Audio and serializes
    /// AudioClip references here so runtime code resolves sound by filename
    /// stem. The asset lives at Assets/_Game/Resources/WorkerAudioCatalog.asset
    /// (Resources keeps runtime lookup build-safe, same idea as ArtCatalog).
    /// &lt;/summary&gt;
    [CreateAssetMenu(fileName = "WorkerAudioCatalog", menuName = "Sy/G11 Audio Catalog")]
    public sealed class WorkerAudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;     // filename stem, e.g. "g11_sfx_salary"
            public AudioClip clip;  // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // The serializer may revive the instance with a stale cache; drop it
            // so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
```

```
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper. Every consumer MUST treat null as "clip missing -&gt;
    /// keep the procedural placeholder" so headless gates stay green on any
    /// project state (catalog absent, audio pruned, name typos) - the same
    /// null-safe law as ArtRuntime.Get.
    /// &lt;/summary&gt;
    public static class WorkerAudioRuntime
    {
        static WorkerAudioCatalog _catalog;
        public static WorkerAudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;WorkerAudioCatalog&gt;("WorkerAudioCatalog");
                return _catalog;
            }
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
    }
}
// WorkerDailyAch.cs
using System;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T010 slice 3a data layer: the daily quest board (3 quests per local
    /// calendar day, requirement doc S04) plus the workplace-meme badge family
    /// ("paid toilet master" / "slack grandmaster" and friends). Pure local
    /// law (red line 12): completion judgment is a pure function of the local
    /// career ledger - zero server, zero wall-clock dependency inside the
    /// engine (the host injects the day key, the check injects fixed ones).
    /// Determinism family law: the per-day quest trio is a pure function of
    /// the day key through an FNV-1a + splitmix64 chain - zero System.Random -
```

```
/// so the same calendar day always deals the same three quests, and a
/// different day rolls a fresh trio (the S04 "refresh assertion").
/// Day rollover law: opening a new day re-arms the stat baselines (quest
/// progress counts only what happened today) and wipes the claim mask
/// (each day's rewards are claimable once per day); re-opening the same
/// day keeps every claim and every baseline exactly as they were.
/// Quest rewards bank through the real sim wallet/exp channels
/// (DepositCoin + GrantExp, the same channels as the offline claim and
/// the red packet); refused paths change zero state. Badges resolve as a
/// pure threshold fold over the lifetime ledger: idempotent, monotone,
/// one bit per badge, never un-earning. The host page, the ledger
/// persistence and the toasts land with the 3b UI slice; DailyAchCheck
/// is the consumer canary.
/// &lt;/summary&gt;
public static class WorkerDailyAch
{
    /// &lt;summary&gt;Requirement doc S04: exactly three daily quests.&lt;/summary&gt;
    public const int DailyQuestCount = 3;
    /// &lt;summary&gt;Career stat streams the quests and badges read. The host
    /// owns the lifetime ledger in the local save; the daily board keeps a
    /// per-day baseline snapshot so quest progress is "since the day
    /// opened", never lifetime totals.&lt;/summary&gt;
    public enum Stat { Slacks = 0, Events = 1, Skills = 2, Wages = 3, Promos = 4, Epics = 5, Retires = 6 }
    public const int StatCount = 7;
    public sealed class QuestDef
    {
        public Stat Stat;
        public int Target;
        public int RewardCoin;
        public int RewardExp;
    }
    // ----- frozen quest pool (9 rows; the day deals 3 distinct picks) -----
    // Targets tune to one play session: wage beats land 1/s, event cards
    // every ~20s, so the top quests ask for minutes of honest grinding.
    public static readonly QuestDef[] QuestPool =
    {
        new QuestDef { Stat = Stat.Slacks, Target = 3,   RewardCoin = 150,  RewardExp = 0   },
        new QuestDef { Stat = Stat.Slacks, Target = 10,  RewardCoin = 400,  RewardExp = 60  },
        new QuestDef { Stat = Stat.Events, Target = 3,   RewardCoin = 200,  RewardExp = 40  },
        new QuestDef { Stat = Stat.Events, Target = 8,   RewardCoin = 500,  RewardExp = 120 },
        new QuestDef { Stat = Stat.Skills, Target = 1,   RewardCoin = 300,  RewardExp = 80  },
        new QuestDef { Stat = Stat.Skills, Target = 3,   RewardCoin = 900,  RewardExp = 200 },
        new QuestDef { Stat = Stat.Wages,  Target = 120, RewardCoin = 250,  RewardExp = 0   },
        new QuestDef { Stat = Stat.Wages,  Target = 600, RewardCoin = 800,  RewardExp = 0   },
        new QuestDef { Stat = Stat.Promos, Target = 1,   RewardCoin = 1000, RewardExp = 200 }
    };
    /// &lt;summary&gt;The lifetime career ledger (local save authority, red line
    /// 12). Plain value struct: byte-parity fingerprints fold the seven
    /// fields directly.&lt;/summary&gt;
    public struct CareerLedger
```

```
{
    public int Slacks;
    public int Events;
    public int Skills;
    public int Wages;
    public int Promos;
    public int Epics;
    public int Retires;
    public int Get(Stat stat)
    {
        switch (stat)
        {
            case Stat.Slacks: return Slacks;
            case Stat.Events: return Events;
            case Stat.Skills: return Skills;
            case Stat.Wages: return Wages;
            case Stat.Promos: return Promos;
            case Stat.Epics: return Epics;
            default: return Retires;
        }
    }
    public void Add(Stat stat, int delta)
    {
        switch (stat)
        {
            case Stat.Slacks: Slacks += delta; break;
            case Stat.Events: Events += delta; break;
            case Stat.Skills: Skills += delta; break;
            case Stat.Wages: Wages += delta; break;
            case Stat.Promos: Promos += delta; break;
            case Stat.Epics: Epics += delta; break;
            default: Retires += delta; break;
        }
    }
    /// &lt;summary&gt;Full snapshot (the daily board baseline).&lt;/summary&gt;
    public int[] Snapshot()
        =&gt; new[] { Slacks, Events, Skills, Wages, Promos, Epics, Retires };
    /// &lt;summary&gt;Fold face for the dual-run parity canary: identical
    /// ledgers always fold to the identical line.&lt;/summary&gt;
    public string Fingerprint()
        =&gt; Slacks + "," + Events + "," + Skills + "," + Wages + ","
         + Promos + "," + Epics + "," + Retires;
}
/// &lt;summary&gt;One calendar day's quest board: the day key it belongs to,
/// the stat baselines frozen at day open (progress = now - baseline)
/// and the once-per-day claim mask (bit i = quest i banked).&lt;/summary&gt;
public sealed class DailyBoard
{
    public string DayKey;
    public readonly int[] Baseline = new int[StatCount];
```

```
    public int ClaimedMask;
    public bool IsValidFor(string dayKey) =&gt; DayKey == dayKey;
    public string Fingerprint()
        =&gt; DayKey + ";" + Baseline[0] + "," + Baseline[1] + "," + Baseline[2] + ","
         + Baseline[3] + "," + Baseline[4] + "," + Baseline[5] + "," + Baseline[6]
         + ";" + ClaimedMask;
}
/// &lt;summary&gt;Day-open law (the S04 refresh assertion): the same day
/// re-opens the exact saved board - claims and baselines untouched;
/// a new day deals a fresh board with the baselines armed at the
/// current ledger and the claim mask wiped, so progress and rewards
/// are strictly per-calendar-day.&lt;/summary&gt;
public static DailyBoard OpenBoard(DailyBoard saved, string dayKey, in CareerLedger ledger)
{
    if (saved != null &amp;&amp; saved.IsValidFor(dayKey)) return saved;
    var board = new DailyBoard();
    board.DayKey = dayKey;
    var snap = ledger.Snapshot();
    Array.Copy(snap, board.Baseline, StatCount);
    board.ClaimedMask = 0;
    return board;
}
/// &lt;summary&gt;The day's three quests: a pure function of the day key
/// (FNV-1a + splitmix64), always three distinct pool rows - zero
/// System.Random, zero clock reads inside the engine. The third slot
/// samples the pool with both taken rows excluded in ascending order:
/// the plain "skip a, then skip b" walk can land back on a when the
/// raw slot sits between b and a (the walk-dup trap the gate caught).
/// &lt;/summary&gt;
public static QuestDef[] QuestsForDay(string dayKey)
{
    ulong h = HashDay(dayKey);
    int n = QuestPool.Length;
    int a = (int)(h % (ulong)n);
    int b = (int)((h &gt;&gt; 21) % (ulong)(n - 1)); if (b &gt;= a) b++;
    int lo = a &lt; b ? a : b;
    int hi = a &lt; b ? b : a;
    int c = (int)((h &gt;&gt; 42) % (ulong)(n - 2));
    if (c &gt;= lo) c++;
    if (c &gt;= hi) c++;
    return new[] { QuestPool[a], QuestPool[b], QuestPool[c] };
}
/// &lt;summary&gt;Quest progress since the day opened: never negative
/// (a mid-session day rollover never poisons the fresh board).&lt;/summary&gt;
public static int Progress(DailyBoard board, in CareerLedger ledger, Stat stat)
{
    int delta = ledger.Get(stat) - board.Baseline[(int)stat];
    return delta &gt; 0 ? delta : 0;
}
public static bool IsComplete(DailyBoard board, in CareerLedger ledger, int questIndex)
```

```
{
    var quests = QuestsForDay(board.DayKey);
    var q = quests[questIndex];
    return Progress(board, in ledger, q.Stat) &gt;= q.Target;
}
/// &lt;summary&gt;Verdict of a quest claim tap.&lt;/summary&gt;
public enum ClaimResult { Granted = 0, Incomplete = 1, AlreadyClaimed = 2 }
/// &lt;summary&gt;Once-per-day claim: banks the quest reward through the
/// real wallet/exp channels exactly once. Refused paths (incomplete or
/// already claimed) change zero state - board, ledger and sim all
/// untouched.&lt;/summary&gt;
public static ClaimResult TryClaim(DailyBoard board, in CareerLedger ledger, int questIndex, WorkerSim sim)
{
    var quests = QuestsForDay(board.DayKey);
    var q = quests[questIndex];
    if ((board.ClaimedMask &amp; (1 &lt;&lt; questIndex)) != 0) return ClaimResult.AlreadyClaimed;
    if (Progress(board, in ledger, q.Stat) &lt; q.Target) return ClaimResult.Incomplete;
    if (sim != null)
    {
        sim.DepositCoin(q.RewardCoin);
        sim.GrantExp(q.RewardExp);
    }
    board.ClaimedMask |= 1 &lt;&lt; questIndex;
    return ClaimResult.Granted;
}
/// &lt;summary&gt;Quest progress line (English source; the host locale face
/// rides the same key through the zh table).&lt;/summary&gt;
public static string QuestLine(DailyBoard board, in CareerLedger ledger, int questIndex)
{
    var q = QuestsForDay(board.DayKey)[questIndex];
    int progress = Progress(board, in ledger, q.Stat);
    // T024-L3 P3 (X1428): a lifetime counter past the small daily target
    // read as an overshoot ("4/1 promotion") - display caps at the
    // target like any progress bar; the claim law above still uses the
    // uncapped Progress, so mechanics are untouched.
    if (progress &gt; q.Target) progress = q.Target;
    return string.Format(WorkerTexts.GetEn(QuestKeyPrefix + q.Stat.ToString().ToLowerInvariant()),
        progress, q.Target);
}
const string QuestKeyPrefix = "quest_kind_";
// ----- the workplace-meme badge family (10 badges, pure thresholds) -----
public const int BadgeCount = 10;
/// &lt;summary&gt;Badge i watches BadgeStat[i] crossing BadgeTarget[i]; the
/// first five are career firsts, the rest are the grind memes (doc S04
/// names: paid toilet master, slack grandmaster).&lt;/summary&gt;
public static readonly Stat[] BadgeStat =
{
    Stat.Wages, Stat.Events, Stat.Skills, Stat.Promos,
    Stat.Slacks, Stat.Slacks, Stat.Events, Stat.Wages, Stat.Epics, Stat.Retires
};
```

```
        public static readonly int[] BadgeTarget = { 1, 1, 1, 1, 25, 100, 50, 600, 1, 1 };
        public static readonly string[] BadgeKey =
        {
            "badge_first_wage", "badge_pie_survivor", "badge_first_skill", "badge_ladder_climber",
            "badge_toilet_master", "badge_slack_grandmaster", "badge_meeting_marathoner",
            "badge_paycheck_collector", "badge_epic_wardrobe", "badge_financial_freedom"
        };
        /// &lt;summary&gt;Pure threshold fold over the lifetime ledger: idempotent,
        /// monotone (a growing ledger never un-earns a badge). The host
        /// persists the mask; re-resolving can only add bits.&lt;/summary&gt;
        public static int ResolveBadges(in CareerLedger ledger)
        {
            int mask = 0;
            for (int i = 0; i &lt; BadgeCount; i++)
                if (ledger.Get(BadgeStat[i]) &gt;= BadgeTarget[i]) mask |= 1 &lt;&lt; i;
            return mask;
        }
        public static string BadgeName(int badgeIndex) =&gt; WorkerTexts.GetEn(BadgeKey[badgeIndex] + "_name");
        public static string BadgeDesc(int badgeIndex) =&gt; WorkerTexts.GetEn(BadgeKey[badgeIndex] + "_desc");
        /// &lt;summary&gt;Quest line key check helper (non-degrading face).&lt;/summary&gt;
        public static string QuestKindKey(Stat stat) =&gt; QuestKeyPrefix + stat.ToString().ToLowerInvariant();
        // ----- day-key hash chain (FNV-1a + splitmix64 avalanche) -----
        static ulong HashDay(string dayKey)
        {
            ulong h = 2166136261u;
            if (!string.IsNullOrEmpty(dayKey))
                for (int i = 0; i &lt; dayKey.Length; i++)
                {
                    h ^= (ulong)dayKey[i];
                    h *= 16777619u;
                }
            return Mix64(h);
        }
        static ulong Mix64(ulong z)
        {
            z = (z ^ (z &gt;&gt; 30)) * 0xBF58476D1CE4E5B9UL;
            z = (z ^ (z &gt;&gt; 27)) * 0x94D049BB133111EBUL;
            return z ^ (z &gt;&gt; 31);
        }
    }
}
// WorkerData.cs
using System;
using System.Globalization;
using System.Text;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// Frozen schema mirror v1 of the G11 numeric truth doc (single source of numbers).
    /// Law: structure moves require SchemaVersion bump; values may be re-tuned by
```

```
/// the bot sweep machine gate (doc re-freezes, this mirror syncs, check re-runs).
/// Every S1 value here is a sweep starting point, not a final number.
/// Section tags below map 1:1 to the doc (wage/dual-axis/skill/gate/offline/adcap/event/pace).
/// &lt;/summary&gt;
public static class WorkerData
{
    public const int SchemaVersion = 1;
    // ----- doc section 1: wage main axis (W01, settles every second online) -----
    public const float BaseWagePerSec = 1.0f;     // intern baseline, coin per second
    public const float ExpPerSecOnline = 1.0f;   // idle exp per second (skill currency)
    public const long CoinCap = 9_999_999;
    public const long ExpCap = 9_999_999;
    public enum WorkerRank { Intern = 0, Corporate = 1, TeamLead = 2, Director = 3, Ceo = 4 }
    /// &lt;summary&gt;Rank coefficient per WorkerRank, monotonic non-decreasing (doc section 1 table).&lt;/summary&
    public static readonly float[] RankCoef = { 1.0f, 1.6f, 2.6f, 4.2f, 7.0f };
    // ----- doc section 2: dual axis of slacking (W03, three segments 0..100) -----
    public const float WorkKpiPerSec = 2.0f;     // Work: KPI up, mood down (mutually exclusive axes)
    public const float WorkMoodPerSec = -1.0f;
    public const float SlackKpiPerSec = -0.5f;   // Slack: mood up, inspiration up, progress down
    public const float SlackMoodPerSec = 2.0f;
    public const float SlackInspPerSec = 1.0f;
    public const float AxisMin = 0f;
    public const float AxisMax = 100f;
    public const float InitKpi = 40f;
    public const float InitMood = 60f;
    public const float InitInsp = 0f;
    public const float MoodStrikeLine = 20f;     // below -&gt; strike show (no lockout, recover path clear)
    public const float MoodRecoverPerSec = 0.5f; // natural regen during strike
    public const float MoodRecoverTo = 25f;      // strike lifts at 25
    public const float KpiWarnLine = 20f;        // below -&gt; layoff warning card
    public const float KpiRecoverWorkSeconds = 10f; // continuous Work lifts the warning
    public const float RarePoolInspGate = 30f;   // inspiration above 30 unlocks the rare event pool
    // ----- doc section 3: comedy skill tree (W02, 4 branches x 3 levels) -----
    public enum SkillBranch { Ppt = 0, Report = 1, SlackArt = 2, ManageUp = 3 }
    public const int SkillLevelCount = 3;
    /// &lt;summary&gt;Upgrade exp cost per level, per branch, monotonic (100/300/800).&lt;/summary&gt;
    public static readonly int[] SkillUpgradeExpCost = { 100, 300, 800 };
    /// &lt;summary&gt;Ppt branch: wage multiplier bonus, wage_mult = 1 + bonus (0.25/0.60/1.20).&lt;/summary&gt;
    public static readonly float[] WageMultBonus = { 0.25f, 0.60f, 1.20f };
    /// &lt;summary&gt;Report branch: KPI gain bonus while working (0.20/0.50/1.00).&lt;/summary&gt;
    public static readonly float[] KpiGainBonus = { 0.20f, 0.50f, 1.00f };
    /// &lt;summary&gt;SlackArt branch: mood gain multiplier while slacking (raw x1.30/x1.75/x2.50, not an additive
    public static readonly float[] SlackMoodMult = { 1.30f, 1.75f, 2.50f };
    /// &lt;summary&gt;SlackArt L3 extra: the KPI loss while slacking is halved.&lt;/summary&gt;
    public const float SlackLossMultAtL3 = 0.5f;
    /// &lt;summary&gt;ManageUp branch: offline rate bonus (0.30/0.75/1.50).&lt;/summary&gt;
    public static readonly float[] OfflineBonus = { 0.30f, 0.75f, 1.50f };
    // ----- doc section 4: promotion gates (W04, dual track: KPI threshold + marker event) -----
    /// &lt;summary&gt;Gate KPI thresholds, monotonic non-decreasing, 4 gates (30/55/80/100).&lt;/summary&gt;
    public static readonly float[] GateKpiThreshold = { 30f, 55f, 80f, 100f };
```

```
/// &lt;summary&gt;Marker event ids aligned gate by gate; gate 4 resolves the dual ending (Ceo run / retire).&lt
public static readonly string[] GateMarkerEventId = { "evt_04", "evt_02b", "evt_12", "evt_09" };
public const float InheritWageMultPerLoop = 0.10f; // NG+ wage multiplier +10% per loop
// ----- doc section 5: offline settlement (W07, zero-server) -----
public const float MaxOfflineHours = 12f;     // min(span, 12h) cap, protects online value
public const float WeekendOfflineMult = 2f;  // Sat/Sun window multiplier
public const int OfflineDoublePerSpan = 1;    // rewarded_double offers per offline span
// ----- doc section 6: AdCap daily hard caps (S02, pure IAA) -----
public enum AdCategory { RewardedDouble = 0, RewardedGacha = 1, RewardedSkip = 2, RewardedBoost = 3 }
/// &lt;summary&gt;Daily cap per AdCategory (5/10/8/6); over-cap precheck refuses with zero state change.&lt;/su
public static readonly int[] AdDailyCap = { 5, 10, 8, 6 };
// ----- doc section 6, T013 point mechanics (design table rows 3-4) -----
/// &lt;summary&gt;Skip point cooldown window (design table: "cooldown-gated"):
/// after a granted skip the point dims for this long. Frozen at 120s so
/// ad-skipping can never outpace the comedy stream's own 20s card cadence
/// - the window is the primary gate, the 8/day cap the safety valve.&lt;/summary&gt;
public const float SkipCooldownSec = 120f;
/// &lt;summary&gt;Boost duration: the granted acceleration window (design table: 30min).&lt;/summary&gt;
public const float BoostDurationSec = 1800f;
/// &lt;summary&gt;Boost wage multiplier while the window is live. Series doubler
/// convention: the offline doubler is x2, the boost rides the same x2.&lt;/summary&gt;
public const float BoostWageMult = 2f;
// ----- doc section 7: workplace event table (W05, data-driven) -----
public enum TriggerKind
{
    Scripted,       // fixed timeline slot (evt_01: first card at T+10s)
    Weekly,         // once per in-game week
    MoodAbove,      // mood &gt; Param
    SlackCount,     // slack sessions counted &gt;= Param
    Idle,           // no operation for &gt;= Param seconds
    Random,         // random pool slot
    KpiOverload,    // KPI &gt; Param sustained while mood &lt; Param2
    StageRandom,    // director stage random
    Annual,         // year-end settlement slot
    RarePool,       // rare pool while inspiration &gt; Param
    SkillUpChance,  // on skill upgrade, chance Param
    StageKpiMet     // director stage with KPI gate met
}
public sealed class EventDef
{
    public string Id;
    public float Kpi, Mood, Insp;          // fx deltas applied on resolve
    public string ChainTo;                // follow-up id, empty = none
    public float ChainChance;             // 0..1, 1 = always chains
    public float FreezeSeconds;           // &gt;0 = freeze show (evt_07)
    public TriggerKind Kind;
    public float Param;                   // primary threshold / count / chance
    public float Param2;                  // secondary threshold
    public bool IsChain;                  // continuation row of a chain
    public float WageMultMin, WageMultMax; // evt_09 riddle range (0..10x daily wage)
```

```
}
public const float EventAvgIntervalSec = 20f;   // avg one card per 20s
public const float EventCooldownSec = 1800f;   // same-id cooldown 30 minutes
public const float FxBoundKpi = 30f;            // |fx.KPI| &lt;= 30
public const float FxBoundMood = 20f;           // |fx.mood| &lt;= 20
public const float FxBoundInsp = 20f;           // |fx.inspiration| &lt;= 20
public static readonly EventDef[] Events =
{
    new EventDef { Id = "evt_01", Kpi = 0f, Mood = 10f, Insp = 0f, ChainTo = "evt_01b", ChainChance = 0.05f, Kin
    new EventDef { Id = "evt_02", Kpi = 0f, Mood = 0f, Insp = 15f, ChainTo = "evt_02b", ChainChance = 1f, Kind =
    new EventDef { Id = "evt_03", Kpi = 0f, Mood = -10f, Insp = 20f, Kind = TriggerKind.MoodAbove, Param = 50f }
    new EventDef { Id = "evt_04", Kpi = -2f, Mood = 15f, Insp = 0f, Kind = TriggerKind.SlackCount, Param = 5f },
    new EventDef { Id = "evt_05", Kpi = 0f, Mood = -5f, Insp = 0f, ChainTo = "evt_05b", ChainChance = 1f, Kind =
    new EventDef { Id = "evt_06", Kpi = 0f, Mood = 2f, Insp = 0f, Kind = TriggerKind.Random },
    new EventDef { Id = "evt_07", Kpi = 0f, Mood = 0f, Insp = 0f, FreezeSeconds = 30f, Kind = TriggerKind.KpiOve
    new EventDef { Id = "evt_08", Kind = TriggerKind.StageRandom },
    new EventDef { Id = "evt_09", Kind = TriggerKind.Annual, WageMultMin = 0f, WageMultMax = 10f },
    new EventDef { Id = "evt_10", Kpi = 0f, Mood = 0f, Insp = 10f, Kind = TriggerKind.RarePool, Param = 30f },
    new EventDef { Id = "evt_11", Kind = TriggerKind.SkillUpChance, Param = 0.10f },
    new EventDef { Id = "evt_12", Kpi = 30f, Mood = 20f, Insp = 0f, Kind = TriggerKind.StageKpiMet },
    new EventDef { Id = "evt_01b", Kpi = 20f, Mood = 0f, Insp = 0f, IsChain = true, Kind = TriggerKind.Scripted
    new EventDef { Id = "evt_02b", Kpi = 0f, Mood = 10f, Insp = 10f, IsChain = true, Kind = TriggerKind.Weekly }
    new EventDef { Id = "evt_05b", Kpi = 0f, Mood = 10f, Insp = 0f, IsChain = true, Kind = TriggerKind.Idle }
};
// ----- doc section 8: pacing anchors (red line 11 window) -----
public const float P1Seconds = 60f;    // first day + first wage
public const float P50Seconds = 300f; // regularized + slack system open + first event burst
// ----- derived faces (data-plane reads, T005 consumes; zero RNG, pure functions) -----
/// &lt;summary&gt;Skill levels learned (0 = not learned) map to bonus tables (L1..L3).&lt;/summary&gt;
static float SkillMult(int levelsLearned, float[] bonusTable)
{
    if (levelsLearned &lt;= 0) return 1f;
    return 1f + bonusTable[ClampIndex(levelsLearned - 1, bonusTable.Length)];
}
static float SkillBonus(int levelsLearned, float[] bonusTable)
    =&gt; levelsLearned &lt;= 0 ? 0f : bonusTable[ClampIndex(levelsLearned - 1, bonusTable.Length)];
/// &lt;summary&gt;Four-factor wage chain: base x rank coef x Ppt skill mult x NG+ inherit mult.&lt;/summary&gt;
public static float WagePerSec(int rankIndex, int pptLevelsLearned, int ngLoopCount)
{
    float rank = RankCoef[ClampIndex(rankIndex, RankCoef.Length)];
    float skill = SkillMult(pptLevelsLearned, WageMultBonus);
    float inherit = 1f + InheritWageMultPerLoop * Math.Max(0, ngLoopCount);
    return BaseWagePerSec * rank * skill * inherit;
}
public static float KpiPerSec(bool working, int reportLevelsLearned, int slackArtLevelsLearned)
{
    if (working) return WorkKpiPerSec + SkillBonus(reportLevelsLearned, KpiGainBonus);
    float lossMult = slackArtLevelsLearned &gt;= SkillLevelCount ? SlackLossMultAtL3 : 1f;
    return SlackKpiPerSec * lossMult;
}
```

```
public static float MoodPerSec(bool working, int slackArtLevelsLearned)
{
    if (working) return WorkMoodPerSec;
    // doc semantics: raw multiplier table (x1.30 at L1), unlearned = x1.0
    float mult = slackArtLevelsLearned &lt;= 0 ? 1f : SlackMoodMult[ClampIndex(slackArtLevelsLearned - 1, SlackM
    return SlackMoodPerSec * mult;
}
public static float InspPerSec(bool working) =&gt; working ? 0f : SlackInspPerSec;
/// &lt;summary&gt;Offline gain: min(span,12h) x online wage rate x ManageUp mult x weekend window.&lt;/summary&
public static double OfflineGainHours(double awayHours, bool weekend, int rankIndex, int pptLevelsLearned, int m
{
    double span = Math.Min(awayHours, MaxOfflineHours);
    double rate = WagePerSec(rankIndex, pptLevelsLearned, ngLoopCount);
    double manage = SkillMult(manageUpLevelsLearned, OfflineBonus);
    return span * 3600.0 * rate * manage * (weekend ? WeekendOfflineMult : 1f);
}
static int ClampIndex(int v, int len) =&gt; Math.Max(0, Math.Min(v, len - 1));
// ----- schema frozen fingerprint (double derive, byte parity is the law) -----
static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
static string F(float v) =&gt; v.ToString("0.###", Inv);
static string Row(EventDef e)
{
    var sb = new StringBuilder();
    sb.Append(e.Id).Append('=')
      .Append(F(e.Kpi)).Append(',').Append(F(e.Mood)).Append(',').Append(F(e.Insp))
      .Append('|').Append(string.IsNullOrEmpty(e.ChainTo) ? "-" : e.ChainTo)
      .Append('@').Append(F(e.ChainChance))
      .Append('|').Append(F(e.FreezeSeconds))
      .Append('|').Append(e.Kind)
      .Append('|').Append(F(e.Param)).Append('/').Append(F(e.Param2))
      .Append("|w").Append(F(e.WageMultMin)).Append('-').Append(F(e.WageMultMax))
      .Append('|').Append(e.IsChain ? "c" : "p");
    return sb.ToString();
}
/// &lt;summary&gt;Derive path A: imperative per-section narration of the frozen tables.&lt;/summary&gt;
public static string DeriveSigA()
{
    var sb = new StringBuilder();
    sb.Append("v").Append(SchemaVersion).Append(';');
    sb.Append("wage:").Append(F(BaseWagePerSec)).Append(',').Append(F(ExpPerSecOnline))
      .Append(',').Append(CoinCap).Append(',').Append(ExpCap).Append(';');
    sb.Append("rank:");
    for (int i = 0; i &lt; RankCoef.Length; i++)
    {
        if (i &gt; 0) sb.Append('|');
        sb.Append(F(RankCoef[i]));
    }
    sb.Append(';');
    sb.Append("axis:").Append(F(WorkKpiPerSec)).Append(',').Append(F(WorkMoodPerSec))
      .Append(',').Append(F(SlackKpiPerSec)).Append(',').Append(F(SlackMoodPerSec))
```

```
  .Append(',').Append(F(SlackInspPerSec))
  .Append('|').Append(F(AxisMin)).Append(',').Append(F(AxisMax))
  .Append('|').Append(F(InitKpi)).Append(',').Append(F(InitMood)).Append(',').Append(F(InitInsp))
  .Append('|').Append(F(MoodStrikeLine)).Append(',').Append(F(MoodRecoverPerSec)).Append(',').Append(F(MoodR
  .Append('|').Append(F(KpiWarnLine)).Append(',').Append(F(KpiRecoverWorkSeconds))
  .Append('|').Append(F(RarePoolInspGate)).Append(';');
sb.Append("skill:");
for (int i = 0; i &lt; SkillUpgradeExpCost.Length; i++)
{
    if (i &gt; 0) sb.Append('|');
    sb.Append(SkillUpgradeExpCost[i]);
}
sb.Append('|');
for (int i = 0; i &lt; WageMultBonus.Length; i++)
{
    if (i &gt; 0) sb.Append('|');
    sb.Append(F(WageMultBonus[i]));
}
sb.Append('|');
for (int i = 0; i &lt; KpiGainBonus.Length; i++)
{
    if (i &gt; 0) sb.Append('|');
    sb.Append(F(KpiGainBonus[i]));
}
sb.Append('|');
for (int i = 0; i &lt; SlackMoodMult.Length; i++)
{
    if (i &gt; 0) sb.Append('|');
    sb.Append(F(SlackMoodMult[i]));
}
sb.Append('|').Append(F(SlackLossMultAtL3)).Append('|');
for (int i = 0; i &lt; OfflineBonus.Length; i++)
{
    if (i &gt; 0) sb.Append('|');
    sb.Append(F(OfflineBonus[i]));
}
sb.Append(';');
sb.Append("gate:").Append(F(GateKpiThreshold[0])).Append('|').Append(F(GateKpiThreshold[1]))
  .Append('|').Append(F(GateKpiThreshold[2])).Append('|').Append(F(GateKpiThreshold[3]))
  .Append('|').Append(GateMarkerEventId[0]).Append(',').Append(GateMarkerEventId[1])
  .Append(',').Append(GateMarkerEventId[2]).Append(',').Append(GateMarkerEventId[3])
  .Append('|').Append(F(InheritWageMultPerLoop)).Append(';');
sb.Append("off:").Append(F(MaxOfflineHours)).Append(',').Append(F(WeekendOfflineMult))
  .Append(',').Append(OfflineDoublePerSpan).Append(';');
sb.Append("adcap:").Append(AdDailyCap[0]).Append(',').Append(AdDailyCap[1])
  .Append(',').Append(AdDailyCap[2]).Append(',').Append(AdDailyCap[3]).Append(';');
sb.Append("evt:");
for (int i = 0; i &lt; Events.Length; i++)
{
    if (i &gt; 0) sb.Append('#');
```

```
            sb.Append(Row(Events[i]));
        }
        sb.Append('|').Append(F(EventAvgIntervalSec)).Append(',').Append(F(EventCooldownSec))
          .Append(',').Append(F(FxBoundKpi)).Append(',').Append(F(FxBoundMood)).Append(',').Append(F(FxBoundInsp)).A
        sb.Append("pace:").Append(F(P1Seconds)).Append(',').Append(F(P50Seconds)).Append(';');
        // Gacha pool freeze (T015 recalibration): the frozen 11-item pool and
        // the pulls-per-ad constant enter the signature so any drift in the
        // reward table flips the fingerprint (X338 recorded pointer).
        sb.Append("gacha:");
        for (int i = 0; i &lt; WorkerGacha.Pool.Length; i++)
        {
            if (i &gt; 0) sb.Append('|');
            sb.Append(WorkerGacha.Pool[i].Id).Append(':')
              .Append((int)WorkerGacha.Pool[i].Tier).Append(':').Append(WorkerGacha.Pool[i].Weight);
        }
        sb.Append('|').Append(WorkerGacha.PullsPerAd);
        return sb.ToString();
    }
    /// &lt;summary&gt;Derive path B: token pipeline over the same frozen tables, joined at the end.&lt;/summary&gt;
    public static string DeriveSigB()
    {
        var sections = new System.Collections.Generic.List&lt;string&gt;
        {
            "v" + SchemaVersion,
            "wage:" + F(BaseWagePerSec) + "," + F(ExpPerSecOnline) + "," + CoinCap + "," + ExpCap,
            "rank:" + string.Join("|", Array.ConvertAll(RankCoef, v =&gt; F(v))),
            "axis:" + F(WorkKpiPerSec) + "," + F(WorkMoodPerSec) + "," + F(SlackKpiPerSec) + "," + F(SlackMoodPerSec
                   + "|" + F(AxisMin) + "," + F(AxisMax)
                   + "|" + F(InitKpi) + "," + F(InitMood) + "," + F(InitInsp)
                   + "|" + F(MoodStrikeLine) + "," + F(MoodRecoverPerSec) + "," + F(MoodRecoverTo)
                   + "|" + F(KpiWarnLine) + "," + F(KpiRecoverWorkSeconds)
                   + "|" + F(RarePoolInspGate),
            "skill:" + string.Join("|", System.Linq.Enumerable.Select(SkillUpgradeExpCost, v =&gt; v.ToString(Inv)))
                    + "|" + string.Join("|", Array.ConvertAll(WageMultBonus, v =&gt; F(v)))
                    + "|" + string.Join("|", Array.ConvertAll(KpiGainBonus, v =&gt; F(v)))
                    + "|" + string.Join("|", Array.ConvertAll(SlackMoodMult, v =&gt; F(v)))
                    + "|" + F(SlackLossMultAtL3)
                    + "|" + string.Join("|", Array.ConvertAll(OfflineBonus, v =&gt; F(v))),
            "gate:" + string.Join("|", Array.ConvertAll(GateKpiThreshold, v =&gt; F(v)))
                   + "|" + string.Join(",", GateMarkerEventId)
                   + "|" + F(InheritWageMultPerLoop),
            "off:" + F(MaxOfflineHours) + "," + F(WeekendOfflineMult) + "," + OfflineDoublePerSpan,
            "adcap:" + string.Join(",", System.Linq.Enumerable.Select(AdDailyCap, v =&gt; v.ToString(Inv))),
            "evt:" + string.Join("#", System.Linq.Enumerable.Select(Events, Row))
                   + "|" + F(EventAvgIntervalSec) + "," + F(EventCooldownSec)
                   + "," + F(FxBoundKpi) + "," + F(FxBoundMood) + "," + F(FxBoundInsp),
            "pace:" + F(P1Seconds) + "," + F(P50Seconds),
            "gacha:" + string.Join("|", System.Linq.Enumerable.Select(WorkerGacha.Pool, c =&gt; c.Id + ":" + (int)c.
                   + "|" + WorkerGacha.PullsPerAd
        };
```

```
            return string.Join(";", sections);
        }
    }
}
// WorkerEventFlow.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T006 workplace event flow engine (SYS04, G08 event-engine paradigm reuse with
    /// zero RNG everywhere): the whole flow is a pure function of the tick stream,
    /// so two identical runs resolve byte-identical cards (EventSig parity law).
    /// Engine laws:
    ///  - opener gate: the scripted T+10s pie meeting (evt_01) owns the first card
    ///    slot (red line 11 first-laugh beat); no other trigger may fire before it.
    ///  - stage-scoped pool: Random / StageRandom / RarePool kinds rotate
    ///    deterministically (index++ % pool.Count, zero RNG), one slot per 240s
    ///    which sits inside the [180,300] pacing band.
    ///  - chains: always-chains land +4s, chance-chains land +8s; a chance like
    ///    0.05 is a deterministic 1-in-20 slot over the resolution counter (no RNG).
    ///  - pacing: non-chain cards keep a 20s minimum gap (EventAvgIntervalSec);
    ///    same-id cards keep the 30-minute cooldown (EventCooldownSec).
    ///  - fx bounds: |kpi|&lt;=30 / |mood|&lt;=20 / |insp|&lt;=20 (FxBound constants).
    /// Rows whose drivers are not live yet in the S1 sim (week cycle, skill
    /// upgrades, annual settlement, director stage) stay armed: the public
    /// On* hooks are called by the later slices once their systems land, so this
    /// engine invents no placeholder behavior for them.
    /// &lt;/summary&gt;
    public sealed class WorkerEventFlow
    {
        public const float PoolRhythmSec = 240f;      // one pool slot per 240s (band [180,300])
        public const float ChainDelayAlwaysSec = 4f;   // always-chains land +4s (paradigm)
        public const float ChainDelayChanceSec = 8f;   // chance-chains land +8s (paradigm)
        public const int ChanceSlotDen = 20;           // 0.05 -&gt; 1-in-20 slots (zero RNG)
        public const float OverloadSustainSec = 5f;     // evt_07 state must be held this long
        public const int DirectorRankIndex = 3;        // WorkerRank.Director stage gate
        /// &lt;summary&gt;Per-tick view of the live sim the triggers read (pure read, no refs).&lt;/summary&gt;
        public struct FlowState
        {
            public float Dt;
            public float SimTime;
            public float Kpi, Mood, Insp;
            public int RankIndex;
            public int SlackSessions;
            public float IdleSeconds;
        }
        /// &lt;summary&gt;One resolved card the host/sim must answer (fx + freeze payload).&lt;/summary&gt;
```

```
public struct ResolvedEvent
{
    public string Id;
    public float Kpi, Mood, Insp;
    public float FreezeSeconds;
    public bool IsChain;
}
public sealed class LogEntry
{
    public string Id;
    public float T;
}
public sealed class PendingChain
{
    public float Due;
    public string Id;
}
static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
readonly List&lt;LogEntry&gt; _log = new List&lt;LogEntry&gt;(64);
readonly Dictionary&lt;string, float&gt; _lastShown = new Dictionary&lt;string, float&gt;();
readonly List&lt;PendingChain&gt; _pending = new List&lt;PendingChain&gt;(8);
readonly Dictionary&lt;string, WorkerData.EventDef&gt; _chainDefs = new Dictionary&lt;string, WorkerData.EventDe
readonly List&lt;WorkerData.EventDef&gt; _conditionDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _poolDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _weeklyDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _annualDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _skillUpDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _stageGateDefs = new List&lt;WorkerData.EventDef&gt;();
readonly List&lt;WorkerData.EventDef&gt; _poolScratch = new List&lt;WorkerData.EventDef&gt;(4);
WorkerData.EventDef _openerDef;
WorkerData.EventDef _overloadDef;
float _nextPoolAt = PoolRhythmSec;
float _nextAllowedAt;
float _overloadSustain;
int _rotationIndex;
int _resolvedTotal;
bool _openerDone;
public float FirstCardT { get; private set; } = -1f;
public int ResolvedTotal =&gt; _resolvedTotal;
public IReadOnlyList&lt;LogEntry&gt; Log =&gt; _log;
public int PendingChainCount =&gt; _pending.Count;
public WorkerEventFlow()
{
    // Kind-driven classification keeps the engine honest against the frozen
    // table: adding/re-tuning rows needs no engine change, only a sig re-run.
    var rows = WorkerData.Events;
    for (int i = 0; i &lt; rows.Length; i++)
    {
        var def = rows[i];
        if (def.IsChain)
```

```
            {
                _chainDefs[def.Id] = def;
                continue;
            }
            switch (def.Kind)
            {
                case WorkerData.TriggerKind.Scripted: _openerDef = def; break;
                case WorkerData.TriggerKind.MoodAbove:
                case WorkerData.TriggerKind.SlackCount:
                case WorkerData.TriggerKind.Idle:
                    _conditionDefs.Add(def); break;
                case WorkerData.TriggerKind.KpiOverload:
                    _overloadDef = def;
                    _conditionDefs.Add(def); break;
                case WorkerData.TriggerKind.Random:
                case WorkerData.TriggerKind.StageRandom:
                case WorkerData.TriggerKind.RarePool:
                    _poolDefs.Add(def); break;
                case WorkerData.TriggerKind.Weekly: _weeklyDefs.Add(def); break;
                case WorkerData.TriggerKind.Annual: _annualDefs.Add(def); break;
                case WorkerData.TriggerKind.SkillUpChance: _skillUpDefs.Add(def); break;
                case WorkerData.TriggerKind.StageKpiMet: _stageGateDefs.Add(def); break;
            }
        }
    }
    /// &lt;summary&gt;Advance the flow one sim tick; resolved cards land in outResolved.&lt;/summary&gt;
    public void Tick(FlowState s, List&lt;ResolvedEvent&gt; outResolved)
    {
        outResolved.Clear();
        // 1) chains first: they are punchline payoffs, exempt from the pacing gap
        for (int i = _pending.Count - 1; i &gt;= 0; i--)
        {
            if (s.SimTime &gt;= _pending[i].Due)
            {
                var def = _chainDefs[_pending[i].Id];
                _pending.RemoveAt(i);
                if (def != null) TryResolve(def, s, outResolved, true);
            }
        }
        // 2) opener: the scripted T+10s pie meeting owns the first-laugh slot
        if (!_openerDone &amp;&amp; _openerDef != null &amp;&amp; s.SimTime &gt;= _openerDef.Param)
        {
            if (TryResolve(_openerDef, s, outResolved, false)) _openerDone = true;
        }
        if (!_openerDone)
        {
            TrackOverload(s);
            return;
        }
        // 3) condition triggers + pool slots respect the pacing gap + cooldowns
```

```
        if (s.SimTime &gt;= _nextAllowedAt)
        {
            for (int i = 0; i &lt; _conditionDefs.Count; i++)
            {
                var def = _conditionDefs[i];
                bool hit;
                switch (def.Kind)
                {
                    case WorkerData.TriggerKind.MoodAbove: hit = s.Mood &gt; def.Param; break;
                    case WorkerData.TriggerKind.SlackCount: hit = s.SlackSessions &gt;= (int)def.Param; break;
                    case WorkerData.TriggerKind.Idle: hit = s.IdleSeconds &gt;= def.Param; break;
                    case WorkerData.TriggerKind.KpiOverload: hit = _overloadSustain &gt;= OverloadSustainSec; break;
                    default: hit = false; break;
                }
                if (hit &amp;&amp; TryResolve(def, s, outResolved, false)) break;
            }
            if (s.SimTime &gt;= _nextPoolAt)
            {
                _nextPoolAt += PoolRhythmSec;
                var pool = BuildPool(s);
                int tries = 0;
                while (tries &lt; pool.Count)
                {
                    var def = pool[_rotationIndex % pool.Count];
                    _rotationIndex++;
                    tries++;
                    if (TryResolve(def, s, outResolved, false)) break;
                }
            }
        }
        TrackOverload(s);
    }
    void TrackOverload(FlowState s)
    {
        bool over = _overloadDef != null &amp;&amp; s.Kpi &gt; _overloadDef.Param &amp;&amp; s.Mood &lt; _overloadDe
        _overloadSustain = over ? _overloadSustain + s.Dt : 0f;
    }
    /// &lt;summary&gt;Stage-scoped rotation pool (deterministic order, zero RNG).&lt;/summary&gt;
    List&lt;WorkerData.EventDef&gt; BuildPool(FlowState s)
    {
        _poolScratch.Clear();
        for (int i = 0; i &lt; _poolDefs.Count; i++)
        {
            var def = _poolDefs[i];
            switch (def.Kind)
            {
                case WorkerData.TriggerKind.Random:
                    _poolScratch.Add(def); break;
                case WorkerData.TriggerKind.StageRandom:
                    if (s.RankIndex &gt;= DirectorRankIndex) _poolScratch.Add(def);
```

```
                break;
            case WorkerData.TriggerKind.RarePool:
                if (s.Insp &gt; def.Param) _poolScratch.Add(def);
                break;
        }
    }
    return _poolScratch;
}
/// &lt;summary&gt;Cooldown + fx bounds + log + chain scheduling. Returns false when
/// the cooldown or an armed guard refused the card (zero state change).&lt;/summary&gt;
bool TryResolve(WorkerData.EventDef def, FlowState s, List&lt;ResolvedEvent&gt; outResolved, bool isChain)
{
    if (def == null) return false;
    if (_lastShown.TryGetValue(def.Id, out float last) &amp;&amp;
        s.SimTime - last &lt; WorkerData.EventCooldownSec) return false;
    _lastShown[def.Id] = s.SimTime;
    _log.Add(new LogEntry { Id = def.Id, T = s.SimTime });
    _resolvedTotal++;
    if (FirstCardT &lt; 0f) FirstCardT = s.SimTime;
    if (!isChain) _nextAllowedAt = s.SimTime + WorkerData.EventAvgIntervalSec;
    outResolved.Add(new ResolvedEvent
    {
        Id = def.Id,
        Kpi = ClampF(def.Kpi, WorkerData.FxBoundKpi),
        Mood = ClampF(def.Mood, WorkerData.FxBoundMood),
        Insp = ClampF(def.Insp, WorkerData.FxBoundInsp),
        FreezeSeconds = def.FreezeSeconds,
        IsChain = isChain
    });
    if (!string.IsNullOrEmpty(def.ChainTo))
    {
        if (def.ChainChance &gt;= 1f)
        {
            Schedule(s.SimTime + ChainDelayAlwaysSec, def.ChainTo);
        }
        else if (SlotHit(ChanceDen(def.ChainChance)))
        {
            Schedule(s.SimTime + ChainDelayChanceSec, def.ChainTo);
        }
    }
    return true;
}
void Schedule(float due, string id)
{
    for (int i = 0; i &lt; _pending.Count; i++)
        if (_pending[i].Id == id) return; // one pending instance per chain id
    _pending.Add(new PendingChain { Due = due, Id = id });
}
/// &lt;summary&gt;Deterministic chance slot: 1-in-den over the resolution counter.&lt;/summary&gt;
bool SlotHit(int den)
```

```
    =&gt; den &lt;= 1 || (_resolvedTotal * 7919 + 17) % den == 0;
static int ChanceDen(float chance)
{
    if (chance &lt;= 0f) return int.MaxValue;
    return Math.Max(1, (int)Math.Round(1.0 / chance));
}
static float ClampF(float v, float bound)
    =&gt; Math.Min(Math.Max(v, -bound), bound);
// ----- later-slice driver hooks (week / skill / annual / stage gate) -----
/// &lt;summary&gt;Week-cycle driver (lands with the week system slice).&lt;/summary&gt;
public void OnWeekPassed(FlowState s, List&lt;ResolvedEvent&gt; outResolved)
{
    if (!_openerDone) return;
    for (int i = 0; i &lt; _weeklyDefs.Count; i++) TryResolve(_weeklyDefs[i], s, outResolved, false);
}
/// &lt;summary&gt;Annual settlement driver (lands with the year/ending slice).&lt;/summary&gt;
public void OnAnnualSettle(FlowState s, List&lt;ResolvedEvent&gt; outResolved)
{
    if (!_openerDone) return;
    for (int i = 0; i &lt; _annualDefs.Count; i++) TryResolve(_annualDefs[i], s, outResolved, false);
}
/// &lt;summary&gt;Skill-upgrade driver; the row chance uses the same deterministic slot law.&lt;/summary&gt;
public void OnSkillUpgraded(FlowState s, List&lt;ResolvedEvent&gt; outResolved)
{
    if (!_openerDone) return;
    for (int i = 0; i &lt; _skillUpDefs.Count; i++)
    {
        var def = _skillUpDefs[i];
        if (SlotHit(ChanceDen(def.Param))) TryResolve(def, s, outResolved, false);
    }
}
/// &lt;summary&gt;Promotion stage-gate driver (director stage with the gate met).&lt;/summary&gt;
public void OnStageGateMet(FlowState s, List&lt;ResolvedEvent&gt; outResolved)
{
    if (!_openerDone || s.RankIndex &lt; DirectorRankIndex) return;
    for (int i = 0; i &lt; _stageGateDefs.Count; i++) TryResolve(_stageGateDefs[i], s, outResolved, false);
}
/// &lt;summary&gt;NG+ roll (T008 doc section 4 inherit law): clears every runtime
/// face - log, cooldowns, pending chains, pool rotation - back to the
/// fresh-boot state so the new career starts with a clean event slate.&lt;/summary&gt;
public void Reset()
{
    _log.Clear();
    _lastShown.Clear();
    _pending.Clear();
    _poolScratch.Clear();
    _nextPoolAt = PoolRhythmSec;
    _nextAllowedAt = 0f;
    _overloadSustain = 0f;
    _rotationIndex = 0;
```

```
            _resolvedTotal = 0;
            _openerDone = false;
            FirstCardT = -1f;
        }
        /// &lt;summary&gt;Invariant event-flow fingerprint: the dual-run byte-parity face
        /// (T005 StateFingerprint law extended with the SYS04 sub-state).&lt;/summary&gt;
        public string EventSig()
        {
            var sb = new StringBuilder(256);
            sb.Append("ev1;first:").Append(FirstCardT.ToString("R", Inv))
              .Append(";n:").Append(_resolvedTotal)
              .Append(";rot:").Append(_rotationIndex)
              .Append(";nextPool:").Append(_nextPoolAt.ToString("R", Inv))
              .Append(";gap:").Append(_nextAllowedAt.ToString("R", Inv))
              .Append(";open:").Append(_openerDone ? 1 : 0)
              .Append(";ovl:").Append(_overloadSustain.ToString("R", Inv))
              .Append(";pend:").Append(_pending.Count);
            for (int i = 0; i &lt; _pending.Count; i++)
                sb.Append('#').Append(_pending[i].Id).Append('@').Append(_pending[i].Due.ToString("R", Inv));
            sb.Append(";cool:").Append(_lastShown.Count);
            var ids = new List&lt;string&gt;(_lastShown.Keys);
            ids.Sort(StringComparer.Ordinal);
            for (int i = 0; i &lt; ids.Count; i++)
                sb.Append('#').Append(ids[i]).Append('@').Append(_lastShown[ids[i]].ToString("R", Inv));
            int tail = _log.Count &gt; 8 ? _log.Count - 8 : 0;
            sb.Append(";logN:").Append(_log.Count).Append(":");
            for (int i = tail; i &lt; _log.Count; i++)
            {
                if (i &gt; tail) sb.Append('#');
                sb.Append(_log[i].Id).Append('@').Append(_log[i].T.ToString("R", Inv));
            }
            return sb.ToString();
        }
    }
}
// WorkerFonts.cs
using UnityEngine;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T024-L2a2 device-truth font resolver (X1420 mine-sweep debt: the About
    /// compliance face renders zh through the built-in LegacyRuntime.ttf which
    /// carries zero CJK glyphs - editor OS fallback masked it, real packages
    /// would tofu, U272 editor-masking law). One shipped subset font covers the
    /// full zh table census (pyftsubset over Texts_zhCN.json non-ASCII set +
    /// margins, OFL license co-copied beside it); every zh face resolves
    /// through here so a missing asset degrades to the built-in fallback
    /// instead of a null font (heads-up in the log, gate still catches it).
    /// &lt;/summary&gt;
    public static class WorkerFonts
```

```
    {
        /// &lt;summary&gt;Resources path of the shipped zh subset (no extension).&lt;/summary&gt;
        public const string ZhFontResource = "Fonts/G11ZhSubset";
        /// &lt;summary&gt;True when the zh subset font asset resolves (gate + wiring probe).&lt;/summary&gt;
        public static bool ZhFontAlive
        {
            get { return Zh() != null; }
        }
        static Font _zh;
        /// &lt;summary&gt;Shipped zh subset font; null when the asset fails to load.&lt;/summary&gt;
        public static Font Zh()
        {
            if (_zh == null) _zh = Resources.Load&lt;Font&gt;(ZhFontResource);
            return _zh;
        }
        /// &lt;summary&gt;Font for a text face: zh faces take the shipped subset,
        /// English faces keep the built-in (it covers ASCII fine).&lt;/summary&gt;
        public static Font Resolve(bool zhFace)
        {
            if (zhFace)
            {
                var f = Zh();
                if (f != null) return f;
                Debug.LogWarning("[WorkerFonts] zh subset font missing, zh face falls back to built-in");
            }
            return Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        }
    }
}
// WorkerGacha.cs
using System;
using System.Collections.Generic;
using Sy.Platform;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T010 slice 1a data layer: the outfit ten-pull gacha engine (W08/S03, pure
    /// IAA - zero IAP, zero real-money gacha; one rewarded ad pays for ten pulls).
    /// Determinism law (same family as WorkerSim): the draw is a pure function of
    /// (seed, pull index) through a splitmix64 integer chain - zero System.Random,
    /// so any two runs with the same seed and the same caller-held state produce
    /// byte-identical pull sequences, which is the O7 canary.
    /// Pity law (series-isomorphic ten-pull guarantee): within one ten-pull, if
    /// the first nine pulls land no Epic, the tenth slot upgrades to a forced
    /// Epic pick - every ad always carries a jackpot, never a dead pull.
    /// AdCap chain (numeric doc section six): rewarded_gacha 10/day through the
    /// shared AdCapPolicy - pre-offer category gate first (honest hint, no dead
    /// ad call), ad verdict second, local cap consume third; an ad fail keeps
    /// the pull retryable with zero state change.
    /// The pool lives here as a frozen data table (requirement doc S03 "pool
```

```
/// table data-driven") and is covered by the WorkerData frozen signature
/// (DeriveSig gacha section - T015 recalibration batch, X338 pointer done).
/// The host page, the real-ad wiring and the share-card face land with the
/// next slices; OfflineCheck O7 is the consumer canary.
/// &lt;/summary&gt;
public static class WorkerGacha
{
    public const string PointId = "g11_gacha_pull";
    public const int PullsPerAd = 10;
    /// &lt;summary&gt;Cosmetic rarity tiers; the pity guarantee is defined over Epic.&lt;/summary&gt;
    public enum Tier { Common = 0, Rare = 1, Epic = 2 }
    public sealed class CosmeticDef
    {
        public string Id;
        public Tier Tier;
        public int Weight;    // relative draw weight within the whole pool
    }
    // ----- frozen pool (11 cosmetics, total weight 100: 60 common / 30 rare / 10 epic) -----
    public static readonly CosmeticDef[] Pool =
    {
        new CosmeticDef { Id = "skin_wrinkled_shirt",      Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "skin_coffee_stain_tie",   Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "skin_free_conf_shirt",    Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "badge_plastic_lanyard",    Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "badge_carpool_sticker",    Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "skin_short_sleeve_suit",   Tier = Tier.Common, Weight = 10 },
        new CosmeticDef { Id = "skin_gym_bro_fit",         Tier = Tier.Rare,   Weight = 10 },
        new CosmeticDef { Id = "skin_silent_spec_auth",    Tier = Tier.Rare,   Weight = 10 },
        new CosmeticDef { Id = "badge_silver_lanyard",     Tier = Tier.Rare,   Weight = 10 },
        new CosmeticDef { Id = "skin_ceo_golden_suit",     Tier = Tier.Epic,   Weight = 5 },
        new CosmeticDef { Id = "badge_penthouse_keycard",  Tier = Tier.Epic,   Weight = 5 }
    };
    static int PoolWeightTotal;
    static int EpicWeightTotal;
    static WorkerGacha()
    {
        foreach (var c in Pool)
        {
            PoolWeightTotal += c.Weight;
            if (c.Tier == Tier.Epic) EpicWeightTotal += c.Weight;
        }
    }
    /// &lt;summary&gt;Caller-held draw state (deterministic, no hidden storage):
    /// the host persists it with the save layer; the check injects it freely.&lt;/summary&gt;
    public struct GachaState
    {
        public int TotalPulls;        // lifetime pulls (seed salt, keeps pulls apart)
        public int PullsSinceEpic;    // distance to the last Epic (stat face)
        public int EpicCount;         // lifetime Epic count (stat face)
    }
```

```
public const double MinClaimSpanSec = 3600.0;
static readonly List&lt;string&gt; FunnelLog = new List&lt;string&gt;(16);
/// &lt;summary&gt;Runtime telemetry sink: the host wires this to
/// Sy.Telemetry (EventIds.Ad with state + pointId + reason fields);
/// the funnel log itself stays the deterministic test canary.&lt;/summary&gt;
public static event Action&lt;string&gt; OnFunnelState;
/// &lt;summary&gt;Ad funnel trail since the last clear ("state:pointId" rows),
/// consumed by OfflineCheck S4.&lt;/summary&gt;
public static IReadOnlyList&lt;string&gt; Funnel =&gt; FunnelLog;
/// &lt;summary&gt;Weekend window law (doc section 5): Saturday/Sunday double.&lt;/summary&gt;
public static bool IsWeekend(DateTime date)
    =&gt; date.DayOfWeek == DayOfWeek.Saturday || date.DayOfWeek == DayOfWeek.Sunday;
/// &lt;summary&gt;Live clock read for the runtime host; the check injects its own
/// stamps so every law stays deterministic in batchmode.&lt;/summary&gt;
public static long NowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
static long ReadStamp()
{
    var raw = UnityEngine.PlayerPrefs.GetString(SeenKey, "");
    return long.TryParse(raw, out var parsed) ? parsed : 0L;
}
/// &lt;summary&gt;Marks the player as seen at now (the stamp authority).&lt;/summary&gt;
public static void Stamp(long nowMs)
{
    UnityEngine.PlayerPrefs.SetString(SeenKey, nowMs.ToString());
    UnityEngine.PlayerPrefs.Save();
}
/// &lt;summary&gt;Test probe: wipes the persisted stamp and the funnel log.&lt;/summary&gt;
public static void ClearForTests()
{
    UnityEngine.PlayerPrefs.DeleteKey(SeenKey);
    FunnelLog.Clear();
}
internal static void Track(string state)
{
    FunnelLog.Add(state + ":" + PointId);
    OnFunnelState?.Invoke(state);
}
/// &lt;summary&gt;Verdict of the once-per-span double offer resolution.&lt;/summary&gt;
public enum DoubleResult { Granted = 0, Blocked = 1, AdFail = 2, Refused = 3 }
/// &lt;summary&gt;
/// One frozen offline claim (P02 OfflineClaim paradigm): the gain freezes at
/// open from the live sim state (rank/Ppt/ManageUp/NG+), collects once -
/// plain or doubled - through the REAL sim wallet, and moves the stamp at
/// collect time. An ignored claim simply re-opens later with more accrual
/// from the same stamp; the once-per-span double offer dies with the grant
/// and survives an ad fail so a retry never wastes the span.
/// &lt;/summary&gt;
public sealed class OfflineClaim
{
    /// &lt;summary&gt;Gain frozen at open (exact double, doc section 5 formula).&lt;/summary&gt;
```

```
public double Gain { get; }
/// &lt;summary&gt;Capped away span in seconds (min(span, 12h); the summary face).&lt;/summary&gt;
public double AwaySec { get; }
/// &lt;summary&gt;Weekend window flag the claim was opened under.&lt;/summary&gt;
public bool Weekend { get; }
/// &lt;summary&gt;Absurd events the flow would have shown (capped span / 20s avg).&lt;/summary&gt;
public int MissedEvents { get; }
/// &lt;summary&gt;True when the away span reached the one-hour claim threshold.&lt;/summary&gt;
public bool Claimable { get; }
/// &lt;summary&gt;True once the claim banked (plain or doubled).&lt;/summary&gt;
public bool Consumed { get; private set; }
/// &lt;summary&gt;True once the span's double offer granted (1/span law).&lt;/summary&gt;
public bool DoubleOfferUsed { get; private set; }
OfflineClaim(double gain, double awaySec, bool weekend, int missed, bool claimable)
{
    Gain = gain;
    AwaySec = awaySec;
    Weekend = weekend;
    MissedEvents = missed;
    Claimable = claimable;
}
/// &lt;summary&gt;Opens the claim for this visit: reads the persisted stamp,
/// freezes the capped span and the gain from the live sim state. A missing
/// stamp (first launch) opens empty and never pays; a short away (or a
/// negative clock skew) re-stamps to now so active play time never
/// accrues offline pay; a claimable span emits the funnel "offer".&lt;/summary&gt;
public static OfflineClaim Open(long nowMs, bool weekend, WorkerSim sim)
{
    long seen = ReadStamp();
    double spanSec = seen &gt; 0L ? Math.Max(0L, nowMs - seen) / 1000.0 : 0.0;
    double cappedSec = Math.Min(spanSec, WorkerData.MaxOfflineHours * 3600.0);
    bool claimable = seen &gt; 0L &amp;&amp; spanSec &gt;= MinClaimSpanSec;
    double gain = 0.0;
    int missed = 0;
    if (claimable)
    {
        int rank = sim != null ? sim.RankIndex : 0;
        int ppt = sim != null ? sim.SkillLevel(WorkerData.SkillBranch.Ppt) : 0;
        int manage = sim != null ? sim.SkillLevel(WorkerData.SkillBranch.ManageUp) : 0;
        int ngLoop = sim != null ? sim.RetireCount : 0;
        gain = WorkerData.OfflineGainHours(cappedSec / 3600.0, weekend, rank, ppt, manage, ngLoop);
        missed = (int)(cappedSec / WorkerData.EventAvgIntervalSec);
    }
    else
    {
        // First launch or a short away: the stamp moves to now, so the
        // next claim counts only true away time after this visit.
        Stamp(nowMs);
    }
    var claim = new OfflineClaim(gain, cappedSec, weekend, missed, claimable);
```

```
    if (claimable) Track("offer");
    return claim;
}
/// &lt;summary&gt;Summary header text (English source; the host locale face
/// lands with the UI slice).&lt;/summary&gt;
public string SummaryHeader() =&gt; WorkerTexts.GetEn("offline_summary_header");
/// &lt;summary&gt;Comedy summary line: away minutes, paid coins, missed absurd
/// events (doc section 5 frozen copy). Deterministic: same claim, same line.&lt;/summary&gt;
public string SummaryLine()
{
    long minutes = (long)Math.Round(AwaySec / 60.0);
    long coins = (long)Math.Round(Gain);
    return string.Format(WorkerTexts.GetEn("offline_summary_line"), minutes, coins, MissedEvents);
}
/// &lt;summary&gt;Collects the claim: banks the frozen gain (or x2 when the
/// rewarded ad granted) through the real sim wallet, stamps the clock at
/// collect time and consumes. Returns the granted coins, or -1 when
/// there is nothing to collect / it was already consumed.&lt;/summary&gt;
public long TryCollect(bool doubled, WorkerSim sim, long nowMs)
{
    if (Consumed || !Claimable || Gain &lt;= 0.0) return -1L;
    double deposit = doubled ? Gain * 2.0 : Gain;
    long granted = (long)Math.Round(deposit);
    if (sim != null) sim.DepositCoin(deposit);
    Stamp(nowMs);
    Consumed = true;
    Track("granted");
    return granted;
}
/// &lt;summary&gt;Once-per-span double offer resolution (numeric doc section
/// 6, rewarded_double 5/day): pre-offer category gate first (the honest
/// hint instead of a dead ad call), then the ad verdict, then the local
/// cap consume - and only a grant burns the span's offer and banks the
/// x2 through the real wallet. Refused paths change zero claim state;
/// a blocked or failed offer leaves the claim collectible as-is.&lt;/summary&gt;
public DoubleResult TryDouble(AdCapPolicy policy, string dayKey, bool adWatched, WorkerSim sim, long nowMs)
{
    if (Consumed || !Claimable || Gain &lt;= 0.0) return DoubleResult.Refused;
    if (DoubleOfferUsed) return DoubleResult.Refused;
    Track("accept");
    string category = AdCategories.CategoryOfPoint(PointId);
    if (policy != null &amp;amp;&amp;amp; !policy.HasCategoryBudget(dayKey, category))
    {
        Track("blocked");
        return DoubleResult.Blocked;
    }
    if (!adWatched)
    {
        Track("ad_fail");
        return DoubleResult.AdFail;
```

```
                }
                if (policy != null &amp;amp;&amp;amp; !policy.TryConsumeRewarded(dayKey, category))
                {
                    Track("blocked");
                    return DoubleResult.Blocked;
                }
                DoubleOfferUsed = true;
                TryCollect(true, sim, nowMs);
                return DoubleResult.Granted;
            }
        }
    }
}
// WorkerPlaceholder.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// Procedural placeholder faces (X027 two-state law): the art catalog stem wins
    /// when real art exists, the deterministic procedural face is the fallback.
    /// Six faces cover the S1 slice: intern unit, four office states, event card frame.
    /// Zero RNG: every pixel is a pure function of (stem, size, palette), so any two
    /// builds are byte-identical and the schema check can assert determinism.
    /// &lt;/summary&gt;
    public static class WorkerPlaceholder
    {
        // Stems match the art requirement doc section 7 work orders (A01/A06-A09/A15).
        public const string StemIntern = "G11_unit_01_intern";
        public const string StemOfficeOpen = "G11_scene_office_open";
        public const string StemOfficeCubicle = "G11_scene_office_cubicle";
        public const string StemOfficePrivate = "G11_scene_office_private";
        public const string StemOfficePenthouse = "G11_scene_office_penthouse";
        public const string StemEventCardFrame = "G11_event_card_frame";
        // T016 P-slice: the remaining nine S1 work-order stems (A02-A05, A10-A14).
        // Badge-tiers (A13) and skills (A14) are multi-cell sheets - the view
        // addresses sliced cells as stem + "_" + index (badge 0..4 by rank,
        // skill 0..3 by branch order Ppt/Report/SlackArt/ManageUp), the stems
        // register now so the catalog keys are final before any art arrives.
        public const string StemUnitBossWang = "G11_unit_02_boss_wang";
        public const string StemUnitHrTian = "G11_unit_03_hr_tian";
        public const string StemUnitKevin = "G11_unit_04_kevin";
        public const string StemUnitMasterZhang = "G11_unit_05_master_zhang";
        public const string StemIconCoin = "G11_icon_ui_coin";
        public const string StemIconMood = "G11_icon_ui_mood";
        public const string StemIconKpi = "G11_icon_ui_kpi";
        public const string StemIconBadgeTiers = "G11_icon_badge_tiers";
        public const string StemIconSkills = "G11_icon_skills";
```

```
/// &lt;summary&gt;Series accent: electric purple (design language anchor).
/// T026 advisory fix (R-20260924-uiaudit-04-g s1.8): brightened from
/// #8B6EF0 (3.36:1 on BaseSlate) to #A489FE so 24px accent text
/// (HookSlogan/Toast) holds WCAG AA &gt;= 4.5:1; hue angle unchanged.&lt;/summary&gt;
public static readonly Color32 AccentElectro = new Color32(0xA4, 0x89, 0xFE, 0xFF);
static readonly Color32 BaseSlate = new Color32(0x2E, 0x33, 0x40, 0xFF);
static readonly Color32 BaseCubicle = new Color32(0x3A, 0x40, 0x4C, 0xFF);
static readonly Color32 BasePrivate = new Color32(0x46, 0x4C, 0x5A, 0xFF);
static readonly Color32 BasePenthouse = new Color32(0x1C, 0x20, 0x2E, 0xFF);
static readonly Color32 BaseCard = new Color32(0x24, 0x28, 0x34, 0xFF);
// T016 P-slice placeholder palettes: one distinct base per new stem so the
// procedural fallback stays visually distinguishable before D-tier art lands.
static readonly Color32 BaseBoss = new Color32(0x3D, 0x33, 0x52, 0xFF);
static readonly Color32 BaseHr = new Color32(0x4A, 0x2E, 0x3C, 0xFF);
static readonly Color32 BaseKevin = new Color32(0x33, 0x3C, 0x33, 0xFF);
static readonly Color32 BaseMaster = new Color32(0x2C, 0x46, 0x40, 0xFF);
static readonly Color32 BaseCoin = new Color32(0x54, 0x44, 0x20, 0xFF);
static readonly Color32 BaseMood = new Color32(0x40, 0x33, 0x4C, 0xFF);
static readonly Color32 BaseKpi = new Color32(0x48, 0x2E, 0x2C, 0xFF);
static readonly Color32 BaseBadge = new Color32(0x50, 0x44, 0x2C, 0xFF);
static readonly Color32 BaseSkill = new Color32(0x2E, 0x3D, 0x52, 0xFF);
static readonly Dictionary&lt;string, Sprite&gt; Cache = new Dictionary&lt;string, Sprite&gt;(StringComparer.Ord
/// &lt;summary&gt;Two-state resolve: catalog sprite first, procedural fallback after (null-safe).&lt;/summary&g
public static Sprite Face(string stem, int size, Color32 baseCol)
{
    lock (Cache)
    {
        if (Cache.TryGetValue(stem, out var cached)) return cached;
    }
    var catalogSprite = ArtRuntime.Get(stem);
    Sprite sprite = catalogSprite != null ? catalogSprite : BuildSprite(Procedural(stem, size, baseCol));
    lock (Cache)
    {
        Cache[stem] = sprite;
    }
    return sprite;
}
/// &lt;summary&gt;Deterministic procedural texture: checker tiles, hash-driven accent stripes, accent border.&l
public static Texture2D Procedural(string stem, int size, Color32 baseCol)
{
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.filterMode = FilterMode.Point;
    tex.wrapMode = TextureWrapMode.Clamp;
    uint h = StemHash(stem);
    int stripeStep = 4 + (int)(h % 5u);                 // 4..8 px accent stripe cadence
    int stripeOffset = (int)((h &gt;&gt; 8) % 8u);
    int tile = 8;                                       // checker tile size
    var px = new Color32[size * size];
    for (int y = 0; y &lt; size; y++)
    {
```

```
            for (int x = 0; x &lt; size; x++)
            {
                var c = (((x / tile) + (y / tile)) &amp; 1) == 0 ? baseCol : Shade(baseCol, 18);
                bool border = x &lt; 2 || y &lt; 2 || x &gt;= size - 2 || y &gt;= size - 2;
                bool stripe = ((x + stripeOffset) % stripeStep) == 0 &amp;&amp; y &lt; size / 4;
                if (border) c = AccentElectro;
                else if (stripe) c = Shade(AccentElectro, -40);
                px[y * size + x] = c;
            }
        }
        tex.SetPixels32(px);
        tex.Apply(false, false);
        return tex;
    }
    /// &lt;summary&gt;All six face stems, in doc order, for the schema check sweep.&lt;/summary&gt;
    public static string[] AllFaceStems() =&gt; new[]
    {
        StemIntern, StemOfficeOpen, StemOfficeCubicle, StemOfficePrivate, StemOfficePenthouse, StemEventCardFrame
    };
    /// &lt;summary&gt;T016: the full fifteen S1 work-order stems in doc order A01-A15
    /// (art requirement doc section 7). The frozen six-face sweep above stays
    /// untouched; this list is the T016 wiring surface for ArtWireCheck.&lt;/summary&gt;
    public static string[] AllT016Stems() =&gt; new[]
    {
        StemIntern, StemUnitBossWang, StemUnitHrTian, StemUnitKevin, StemUnitMasterZhang,
        StemOfficeOpen, StemOfficeCubicle, StemOfficePrivate, StemOfficePenthouse,
        StemIconCoin, StemIconMood, StemIconKpi, StemIconBadgeTiers, StemIconSkills,
        StemEventCardFrame
    };
    /// &lt;summary&gt;Base color per face (office states get progressively darker/tidier palettes).&lt;/summary&gt;
    public static Color32 BaseColorFor(string stem)
    {
        if (stem == StemOfficeOpen) return BaseSlate;
        if (stem == StemOfficeCubicle) return BaseCubicle;
        if (stem == StemOfficePrivate) return BasePrivate;
        if (stem == StemOfficePenthouse) return BasePenthouse;
        if (stem == StemEventCardFrame) return BaseCard;
        if (stem == StemUnitBossWang) return BaseBoss;
        if (stem == StemUnitHrTian) return BaseHr;
        if (stem == StemUnitKevin) return BaseKevin;
        if (stem == StemUnitMasterZhang) return BaseMaster;
        if (stem == StemIconCoin) return BaseCoin;
        if (stem == StemIconMood) return BaseMood;
        if (stem == StemIconKpi) return BaseKpi;
        if (stem == StemIconBadgeTiers) return BaseBadge;
        if (stem == StemIconSkills) return BaseSkill;
        return BaseSlate;
    }
    static Sprite BuildSprite(Texture2D tex)
        =&gt; Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), new Vector2(0.5f, 0.5f), 100f, 0, SpriteMesh
```

```
        static Color32 Shade(Color32 c, int d)
            =&gt; new Color32(ClampByte(c.r + d), ClampByte(c.g + d), ClampByte(c.b + d), c.a);
        static byte ClampByte(int v) =&gt; (byte)Mathf.Clamp(v, 0, 255);
        static uint StemHash(string stem)
        {
            uint h = 2166136261u;
            for (int i = 0; i &lt; stem.Length; i++)
            {
                h ^= stem[i];
                h *= 16777619u;
            }
            return h;
        }
    }
}
// WorkerSaveData.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using UnityEngine;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T012 data-layer frozen mirror of the local career save (red line 12: pure
    /// local authority, zero server). Field face = requirements doc section 10:
    /// coin / exp / rank / kpi / mood / insp / 4 skill levels / events-seen set /
    /// offline timestamp / NG+ inheritance slot (ngLoop + coin-keep law). Two
    /// determinism carries (simTime + wageBeatCarry) ride along so a resumed run
    /// continues the same beat/op stream (resume-the-journey law).
    /// Host-side saves (g11_gacha_* / g11_career_*) stay in PlayerPrefs and merge
    /// through a READ-ONLY loader with the Host wiring slice - this data layer
    /// owns zero PlayerPrefs write paths (the load zero-overwrite guard shape).
    /// Corrupt or schema-mismatched JSON refuses to load and answers the
    /// fresh-start face: never a crash, never a partial apply.
    /// &lt;/summary&gt;
    [Serializable]
    public sealed class WorkerSaveData
    {
        public const int MirrorSchemaVersion = 1;
        public int schemaVersion = MirrorSchemaVersion;
        public double coin;             // wallet (survives the NG+ retire: coin-keep law)
        public double exp;              // skill currency
        public int rankIndex;           // career rank
        public int ngLoop;              // NG+ inheritance slot (wage x(1+0.10/loop))
        public int retireCount;         // retire history (carries across loops)
        public float kpi;               // dual axis 1
        public float mood;              // dual axis 2
        public float insp;              // inspiration axis
        public bool working = true;     // work/slack mode
```

```
public double simTime;          // determinism carry: dt-stream resume point
public double wageBeatCarry;    // determinism carry: settle-axis beat carry
public int[] skillLevels = new int[4];
public string[] eventsSeen = Array.Empty&lt;string&gt;();
public long offlineUnix;         // wall-clock stamp (offline settle anchor)
/// &lt;summary&gt;Capture the frozen mirror off a live sim. Single source: the
/// sim owns the whole live face; the distinct resolved event ids fold the
/// events-seen set out of the flow log.&lt;/summary&gt;
public static WorkerSaveData Capture(WorkerSim sim, long offlineUnix)
{
    var save = new WorkerSaveData
    {
        schemaVersion = MirrorSchemaVersion,
        coin = sim.Coin,
        exp = sim.Exp,
        rankIndex = sim.RankIndex,
        ngLoop = sim.NgLoop,
        retireCount = sim.RetireCount,
        kpi = sim.Kpi,
        mood = sim.Mood,
        insp = sim.Insp,
        working = sim.Working,
        simTime = sim.SimTime,
        wageBeatCarry = sim.WageBeatCarry,
        skillLevels = new[]
        {
            sim.SkillLevel(WorkerData.SkillBranch.Ppt),
            sim.SkillLevel(WorkerData.SkillBranch.Report),
            sim.SkillLevel(WorkerData.SkillBranch.SlackArt),
            sim.SkillLevel(WorkerData.SkillBranch.ManageUp)
        },
        offlineUnix = offlineUnix
    };
    var seen = new List&lt;string&gt;();
    var log = sim.EventLog;
    for (int i = 0; i &lt; log.Count; i++)
    {
        string id = log[i].Id;
        if (!seen.Contains(id)) seen.Add(id);
    }
    save.eventsSeen = seen.ToArray();
    return save;
}
public string ToJson() =&gt; JsonUtility.ToJson(this);
/// &lt;summary&gt;Load law: corrupt / empty / schema-mismatched JSON answers the
/// fresh-start face (never null, never throws to the caller).&lt;/summary&gt;
public static WorkerSaveData FromJsonOrFresh(string json)
{
    if (!string.IsNullOrEmpty(json))
    {
```

```
        try
        {
            var save = JsonUtility.FromJson&lt;WorkerSaveData&gt;(json);
            if (save != null &amp;&amp; save.schemaVersion == MirrorSchemaVersion
                &amp;&amp; save.skillLevels != null &amp;&amp; save.skillLevels.Length == 4)
                return save;
        }
        catch (System.Exception) { /* corrupt payload: fall through to fresh */ }
    }
    return FreshStart();
}
/// &lt;summary&gt;The fresh-start face: identical to a brand-new WorkerSim
/// constructor face (init axes, wallet empty, intern rank, loop zero).&lt;/summary&gt;
public static WorkerSaveData FreshStart()
{
    return new WorkerSaveData
    {
        kpi = WorkerData.InitKpi,
        mood = WorkerData.InitMood,
        insp = WorkerData.InitInsp,
        working = true,
        skillLevels = new int[4],
        eventsSeen = Array.Empty&lt;string&gt;()
    };
}
/// &lt;summary&gt;Full-mirror round-trip fingerprint (field-by-field canary).&lt;/summary&gt;
public string FaceFingerprint()
{
    var sb = new StringBuilder();
    sb.Append("v").Append(schemaVersion).Append(';');
    sb.Append("coin:").Append(coin.ToString("R", Inv)).Append(';');
    sb.Append("exp:").Append(exp.ToString("R", Inv)).Append(';');
    sb.Append("rank:").Append(rankIndex).Append(",ng:").Append(ngLoop)
      .Append(",ret:").Append(retireCount).Append(';');
    sb.Append("axis:").Append(kpi.ToString("R", Inv)).Append(',')
      .Append(mood.ToString("R", Inv)).Append(',')
      .Append(insp.ToString("R", Inv)).Append(';');
    sb.Append("mode:").Append(working ? 1 : 0).Append(';');
    sb.Append("t:").Append(simTime.ToString("R", Inv)).Append(',')
      .Append(wageBeatCarry.ToString("R", Inv)).Append(';');
    sb.Append("skills:").Append(SkillsText()).Append(';');
    sb.Append("seen:").Append(SeenText()).Append(';');
    sb.Append("off:").Append(offlineUnix);
    return sb.ToString();
}
/// &lt;summary&gt;Core-face fingerprint (mirror minus the events-seen fold): the
/// restore idempotence canary - the flow slate re-arms fresh after a
/// restore, so the seen set is a point-in-time fold, not a restored face.&lt;/summary&gt;
public string CoreFaceFingerprint()
{
```

```
            var sb = new StringBuilder();
            sb.Append("v").Append(schemaVersion).Append(';');
            sb.Append("coin:").Append(coin.ToString("R", Inv)).Append(';');
            sb.Append("exp:").Append(exp.ToString("R", Inv)).Append(';');
            sb.Append("rank:").Append(rankIndex).Append(",ng:").Append(ngLoop)
              .Append(",ret:").Append(retireCount).Append(';');
            sb.Append("axis:").Append(kpi.ToString("R", Inv)).Append(',')
              .Append(mood.ToString("R", Inv)).Append(',')
              .Append(insp.ToString("R", Inv)).Append(';');
            sb.Append("mode:").Append(working ? 1 : 0).Append(';');
            sb.Append("t:").Append(simTime.ToString("R", Inv)).Append(',')
              .Append(wageBeatCarry.ToString("R", Inv)).Append(';');
            sb.Append("skills:").Append(SkillsText()).Append(';');
            sb.Append("off:").Append(offlineUnix);
            return sb.ToString();
        }
        string SkillsText()
            =&gt; skillLevels == null ? "null" : string.Join(",", skillLevels);
        string SeenText()
            =&gt; eventsSeen == null ? "null" : string.Join("|", eventsSeen);
        static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
    }
}
// WorkerSaveWire.cs
using UnityEngine;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T012 slice 2 host wiring: the unified save slot (red line 12: pure local
    /// authority, zero server). The frozen mirror round-trips through exactly
    /// one PlayerPrefs key; the host-side faces (g11_gacha_* / g11_career_*)
    /// stay outside this wiring - the load zero-overwrite guard: this class
    /// never writes another face's keys, so a resume can never clobber the
    /// gacha chain or the career ledger.
    /// Save bookends (quit / destroy / pause / promote / retire) flush through
    /// SaveSlot; Awake discriminates a returning player through HasRealSave (a
    /// wall-clock stamp only a real save carries) so the day-one tutorial arms
    /// for new players only.
    /// &lt;/summary&gt;
    public static class WorkerSaveWire
    {
        const string MirrorPrefsKey = "g11_save_mirror";
        /// &lt;summary&gt;Unified load: the mirror JSON answers the fresh-start face
        /// on a missing or corrupt slot (never null, never throws).&lt;/summary&gt;
        public static WorkerSaveData LoadSlot()
            =&gt; WorkerSaveData.FromJsonOrFresh(PlayerPrefs.GetString(MirrorPrefsKey, ""));
        /// &lt;summary&gt;Unified save: one capture, one key, one flush.&lt;/summary&gt;
        public static void SaveSlot(WorkerSim sim)
        {
            var save = WorkerSaveData.Capture(sim, WorkerOffline.NowMs());
```

```
            PlayerPrefs.SetString(MirrorPrefsKey, save.ToJson());
            PlayerPrefs.Save();
        }
        /// &lt;summary&gt;Returning-player discrimination: only a real save carries a
        /// wall-clock offline stamp (the fresh face ships offlineUnix = 0), so
        /// the tutorial arms exactly once - on the true first run.&lt;/summary&gt;
        public static bool HasRealSave(WorkerSaveData save)
            =&gt; save != null &amp;&amp; save.offlineUnix &gt; 0L;
        /// &lt;summary&gt;Test probe: wipes the mirror slot only (sentinels and the
        /// host-side faces belong to their callers).&lt;/summary&gt;
        public static void ClearSlotForTests()
        {
            PlayerPrefs.DeleteKey(MirrorPrefsKey);
            PlayerPrefs.Save();
        }
    }
}
// WorkerShareSpecs.cs
using System;
using System.Globalization;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T010 slice 2: the auto-composed show-off card specs (doc section 8: the
    /// offline summary data - away span / paid coins / missed events - is the
    /// brag text source). Win-only law, series-isomorphic with the P02 share
    /// specs: an unclaimable or empty claim never brags (null), a claimable
    /// claim brags the frozen numbers - never a fabricated figure. The card
    /// face stays procedural until the art batch lands a template stem
    /// (template key null walks the ShareCardService fallback bg, headless
    /// green); the compose budget ladder and the pixel-readback canary live
    /// in the shared Sy.Social service and ShareCardCheck.
    /// &lt;/summary&gt;
    public static class WorkerShareSpecs
    {
        public const string Footer = "Crazy Worker - Idle Office Life";
        public const string OfflineTitleKey = "share_offline_title";
        public const string OfflineCoinsKey = "share_offline_line_coins";
        public const string OfflineEventsKey = "share_offline_line_events";
        public const string OfflineHookKey = "share_offline_line_hook";
        /// &lt;summary&gt;Brag spec from an open offline claim, or null when the claim
        /// never reached the one-hour threshold or carries no gain (a short
        /// visit is not a brag). Deterministic: the same claim composes the
        /// same copy byte-for-byte on every call.&lt;/summary&gt;
        public static Sy.Social.ShareCard FromOfflineClaim(WorkerOffline.OfflineClaim claim)
        {
            if (claim == null || !claim.Claimable || claim.Gain &lt;= 0.0) return null;
            long minutes = (long)Math.Round(claim.AwaySec / 60.0);
            long coins = (long)Math.Round(claim.Gain);
            return Sy.Social.AutoCardComposer.Card(null,
```

```
                string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineTitleKey), minutes),
                new[]
                {
                    string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineCoinsKey), coins),
                    string.Format(CultureInfo.InvariantCulture, WorkerTexts.GetEn(OfflineEventsKey), claim.MissedEvents)
                    WorkerTexts.GetEn(OfflineHookKey)
                },
                Footer,
                "src=share&amp;span=" + minutes.ToString(CultureInfo.InvariantCulture));
        }
    }
}
// WorkerSim.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Security.Cryptography;
using System.Text;
using Sy.Platform;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// T005 wage main axis simulation core (pure C#, zero RNG, no scene dependency).
    /// Determinism law: state is a pure function of the (dt stream, op stream), so any
    /// two runs with identical inputs produce byte-identical state fingerprints -
    /// which is the offline/online isomorphism law of the numeric doc section 1.
    /// Red line 11 anchors owned by this slice: hook content live from frame zero,
    /// first wage beat lands within 1.5s with zero player ops (zero login wall),
    /// the beat carries the number+pulse+sound three-sync contract, and the Work/Slack
    /// toggle answers from t=0 with no unlock gate in front.
    /// T006-a extension: the sim hosts the SYS04 WorkerEventFlow - it feeds the flow
    /// a per-tick state view, applies the resolved card fx (clamped axes), owns the
    /// forced-rest freeze window (evt_07) and folds the flow EventSig into the state
    /// fingerprint so the dual-run byte-parity law covers the event face too.
    /// &lt;/summary&gt;
    public sealed class WorkerSim
    {
        /// &lt;summary&gt;Wage settle cadence: one beat per second (numeric doc section 1, W01).&lt;/summary&gt;
        public const float WageBeatPeriodSec = 1f;
        // ----- T007 comedy skill tree (doc section 3): the red-line-11 first-skill beat -----
        /// &lt;summary&gt;Tutorial guide slot: inside the T+3-10s window of the per-second table.
        /// The day-one red packet carries exactly one L1 cost (100 exp, doc section 3),
        /// so a fresh hire can land the first skill upgrade inside the 8s bound.&lt;/summary&gt;
        public const float TutorialGuideAtSec = 5f;
        public const double TutorialRedPacketExp = 100.0;
        /// &lt;summary&gt;Upgrade verdict for the skill face: Ok / NoExp (refused, zero state
        /// change) / Maxed (level cap reached, zero state change).&lt;/summary&gt;
        public enum SkillUpResult { Ok = 0, NoExp = 1, Maxed = 2 }
        // ----- live state -----
        double _coin;
```

```
double _exp;
double _simTime;
double _wageBeatCarry;
float _kpi;
float _mood;
float _insp;
bool _working = true;
int _rankIndex;
int _ngLoop;
readonly int[] _skillLevels = new int[4];
// ----- T008: dual-axis flip shows + promotion gates + NG+ endings -----
bool _strike;
int _strikeCount;
bool _warn;
int _warnCount;
float _warnWorkSeconds;
int _gateMarkerMask;
int _promoteOfferMask;
float _firstPromoteT = -1f;
int _promoteCount;
EndingChoice _ending;
int _retireCount;
// ----- red line 11 timing capture -----
public float HookReadyT { get; private set; } = -1f;
public float FirstWageT { get; private set; } = -1f;
public float FirstOpT { get; private set; } = -1f;
public bool FirstWageThreeSync { get; private set; }
public int WageBeatCount { get; private set; }
// ----- T007 skill tree + tutorial face -----
bool _tutorial;
bool _tutorialGuideDone;
float _tutorialGuideT = -1f;
public float FirstSkillUpT { get; private set; } = -1f;
public int SkillUpgradeCount { get; private set; }
public bool TutorialGuideDone =&gt; _tutorialGuideDone;
public float TutorialGuideT =&gt; _tutorialGuideT;
/// &lt;summary&gt;T007: fires exactly once when the day-one skill guide beat lands.&lt;/summary&gt;
public event Action&lt;float&gt; OnTutorialGuide;
// ----- T006-a: SYS04 workplace event flow -----
readonly WorkerEventFlow _flow = new WorkerEventFlow();
readonly List&lt;WorkerEventFlow.ResolvedEvent&gt; _resolvedScratch = new List&lt;WorkerEventFlow.ResolvedEvent&
int _slackSessions;
float _idleSeconds;
float _freezeRemaining;
/// &lt;summary&gt;
/// One settle beat. The presentation must answer THIS SAME beat with all three
/// faces in the same frame: the readout number, the pulse punch and the sound.
/// (The rendered-frame and audible co-hit claims belong to the RT slice; the
/// sim owns the contract, WorkerMainHost consumes it.)
/// &lt;/summary&gt;
```

```
public struct WageBeat
{
    public float T;
    public double Amount;
    public double CoinAfter;
    public bool Number;
    public bool Pulse;
    public bool Sound;
}
public event Action&lt;WageBeat&gt; OnWageBeat;
/// &lt;summary&gt;T006-a: fires the same tick a workplace event card resolves.&lt;/summary&gt;
public event Action&lt;WorkerEventFlow.ResolvedEvent&gt; OnEventCard;
/// &lt;summary&gt;T008-b2: fires when the flip-show tuple changes inside a tick;
/// (striking, warned) is the live show state - the host owns the card face
/// and the recovery toasts (a show, never a lockout).&lt;/summary&gt;
public event Action&lt;bool, bool&gt; OnFlipShow;
/// &lt;summary&gt;T008-b2: fires the same tick a promotion lands (new rank index),
/// after the ceremony-card drive so the host trio (red dot + horn + badge
/// swap) lands on the same frame as the promotion itself.&lt;/summary&gt;
public event Action&lt;int&gt; OnPromoted;
// ----- T010 slice 2: promotion funnel (doc section 8 evt_promote) -----
static readonly List&lt;string&gt; PromoteFunnelLog = new List&lt;string&gt;(16);
/// &lt;summary&gt;Runtime telemetry sink: the host wires this to Sy.Telemetry
/// (EventIds.Promote with state + game fields); the funnel log itself
/// stays the deterministic check canary.&lt;/summary&gt;
public static event Action&lt;string&gt; OnPromoteFunnel;
/// &lt;summary&gt;Promotion funnel trail since the last clear
/// ("state:g11_promote" rows), consumed by ShareCardCheck.&lt;/summary&gt;
public static IReadOnlyList&lt;string&gt; PromoteFunnel =&gt; PromoteFunnelLog;
/// &lt;summary&gt;Test probe: wipes the promotion funnel log only.&lt;/summary&gt;
public static void ClearPromoteFunnelForTests() =&gt; PromoteFunnelLog.Clear();
static void TrackPromote(string state)
{
    PromoteFunnelLog.Add(state + ":g11_promote");
    OnPromoteFunnel?.Invoke(state);
}
// ----- T013: the two remaining IAA points (doc section 6 rows 3-4) -----
/// &lt;summary&gt;Event-card skip point id (funnel + AdCap namespace, g11_-isolated).&lt;/summary&gt;
public const string SkipPointId = "g11_event_skip";
/// &lt;summary&gt;30-minute wage boost point id (funnel + AdCap namespace, g11_-isolated).&lt;/summary&gt;
public const string BoostPointId = "g11_boost_30m";
float _skipCooldownRemaining;
float _boostRemaining;
int _skipCount;
int _boostCount;
static readonly List&lt;string&gt; IaaFunnelLog = new List&lt;string&gt;(32);
/// &lt;summary&gt;Runtime telemetry sink for both T013 points: the host wires
/// this to Sy.Telemetry (EventIds.Ad with state + point + game fields);
/// the funnel log itself stays the deterministic O8/O9 canary.&lt;/summary&gt;
public static event Action&lt;string, string&gt; OnIaaFunnel;
```

```
/// &lt;summary&gt;Ad funnel trail since the last clear ("state:pointId" rows),
/// consumed by OfflineCheck O8/O9.&lt;/summary&gt;
public static IReadOnlyList&lt;string&gt; IaaFunnel =&gt; IaaFunnelLog;
/// &lt;summary&gt;Test probe: wipes the T013 funnel log only.&lt;/summary&gt;
public static void ClearIaaFunnelForTests() =&gt; IaaFunnelLog.Clear();
static void TrackIaa(string state, string pointId)
{
    IaaFunnelLog.Add(state + ":" + pointId);
    OnIaaFunnel?.Invoke(state, pointId);
}
/// &lt;summary&gt;Verdict of the ad-gated event-skip resolution.&lt;/summary&gt;
public enum SkipResult { Granted = 0, Blocked = 1, AdFail = 2, InWindow = 3 }
/// &lt;summary&gt;Verdict of the ad-gated wage-boost resolution.&lt;/summary&gt;
public enum BoostResult { Granted = 0, Blocked = 1, AdFail = 2, InWindow = 3 }
/// &lt;summary&gt;Skip point live window face: true while the cooldown gate
/// holds (the host dims the button - the dim IS the honest hint).&lt;/summary&gt;
public bool SkipInCooldown =&gt; _skipCooldownRemaining &gt; 0f;
public float SkipCooldownRemaining =&gt; _skipCooldownRemaining;
public int SkipCount =&gt; _skipCount;
/// &lt;summary&gt;Boost point live window face: true while the x2 wage
/// acceleration runs.&lt;/summary&gt;
public bool BoostActive =&gt; _boostRemaining &gt; 0f;
public float BoostRemaining =&gt; _boostRemaining;
public int BoostCount =&gt; _boostCount;
/// &lt;summary&gt;
/// T013 event-skip resolution (doc section 6 row 3, rewarded_skip 8/day
/// cooldown-gated): pre-offer window gate first (inside the cooldown the
/// offer never opens - zero state, zero funnel, the dimmed button is the
/// law), then the honest category hint, then the ad verdict, then the
/// local cap consume - and only a grant resolves. A grant ends the
/// current event's show at once (an armed freeze window lifts) and arms
/// the skip cooldown; scheduled chains keep their own beats per rules.
/// Every refused path changes zero state.
/// &lt;/summary&gt;
public SkipResult TrySkip(AdCapPolicy policy, string dayKey, bool adWatched)
{
    if (_skipCooldownRemaining &gt; 0f) return SkipResult.InWindow;
    TrackIaa("offer", SkipPointId);
    TrackIaa("accept", SkipPointId);
    string category = AdCategories.CategoryOfPoint(SkipPointId);
    if (policy != null &amp;&amp; !policy.HasCategoryBudget(dayKey, category))
    {
        TrackIaa("blocked", SkipPointId);
        return SkipResult.Blocked;
    }
    if (!adWatched)
    {
        TrackIaa("ad_fail", SkipPointId);
        return SkipResult.AdFail;
    }
```

```
    if (policy != null &amp;amp;&amp;amp; !policy.TryConsumeRewarded(dayKey, category))
    {
        TrackIaa("blocked", SkipPointId);
        return SkipResult.Blocked;
    }
    _freezeRemaining = 0f;
    _skipCooldownRemaining = WorkerData.SkipCooldownSec;
    _skipCount++;
    TrackIaa("granted", SkipPointId);
    return SkipResult.Granted;
}
/// &lt;summary&gt;
/// T013 wage-boost resolution (doc section 6 row 4, rewarded_boost 6/day):
/// pre-offer window gate first (a repeat tap while the x2 runs refuses
/// pre-offer with zero state change), then the honest category hint, then
/// the ad verdict, then the local cap consume - and only a grant arms the
/// 30-minute x2 wage window. Every refused path changes zero state.
/// &lt;/summary&gt;
public BoostResult TryBoost(AdCapPolicy policy, string dayKey, bool adWatched)
{
    if (_boostRemaining &gt; 0f) return BoostResult.InWindow;
    TrackIaa("offer", BoostPointId);
    TrackIaa("accept", BoostPointId);
    string category = AdCategories.CategoryOfPoint(BoostPointId);
    if (policy != null &amp;amp;&amp;amp; !policy.HasCategoryBudget(dayKey, category))
    {
        TrackIaa("blocked", BoostPointId);
        return BoostResult.Blocked;
    }
    if (!adWatched)
    {
        TrackIaa("ad_fail", BoostPointId);
        return BoostResult.AdFail;
    }
    if (policy != null &amp;amp;&amp;amp; !policy.TryConsumeRewarded(dayKey, category))
    {
        TrackIaa("blocked", BoostPointId);
        return BoostResult.Blocked;
    }
    _boostRemaining = WorkerData.BoostDurationSec;
    _boostCount++;
    TrackIaa("granted", BoostPointId);
    return BoostResult.Granted;
}
/// &lt;summary&gt;T013: external card injection - an engine-resolved card from
/// outside the tick stream (machine-gate driver, LiveOps seat) travels
/// the SAME fx/freeze/card application path a tick card travels, so
/// injected cards behave exactly like tick cards.&lt;/summary&gt;
public void NotifyExternalCard(WorkerEventFlow.ResolvedEvent card)
{
```

```
    _resolvedScratch.Clear();
    _resolvedScratch.Add(card);
    ApplyResolved(_resolvedScratch);
}
public double Coin =&gt; _coin;
public double Exp =&gt; _exp;
public double SimTime =&gt; _simTime;
public double WageBeatCarry =&gt; _wageBeatCarry;
public int NgLoop =&gt; _ngLoop;
public float Kpi =&gt; _kpi;
public float Mood =&gt; _mood;
public float Insp =&gt; _insp;
public bool Working =&gt; _working;
public int RankIndex =&gt; _rankIndex;
public int SlackSessions =&gt; _slackSessions;
public float IdleSeconds =&gt; _idleSeconds;
public float FreezeRemaining =&gt; _freezeRemaining;
public float FirstEventCardT =&gt; _flow.FirstCardT;
public IReadOnlyList&lt;WorkerEventFlow.LogEntry&gt; EventLog =&gt; _flow.Log;
// ----- T008 promotion / flip-show face (slice a: sim law, host lands with slice b) -----
public bool IsStriking =&gt; _strike;
public int StrikeCount =&gt; _strikeCount;
public bool IsWarned =&gt; _warn;
public int WarnCount =&gt; _warnCount;
public float WarnWorkSeconds =&gt; _warnWorkSeconds;
public int GateMarkerMask =&gt; _gateMarkerMask;
public float FirstPromoteT =&gt; _firstPromoteT;
public int PromoteCount =&gt; _promoteCount;
public EndingChoice CurrentEnding =&gt; _ending;
public int RetireCount =&gt; _retireCount;
public bool EndingChoiceOpen
    =&gt; _rankIndex &gt;= (int)WorkerData.WorkerRank.Ceo &amp;&amp; _ending == EndingChoice.None;
public WorkerSim() : this(false) { }
/// &lt;summary&gt;Tutorial mode is opt-in so the frozen gate scenarios (Redline11Check
/// main run, EventEngineCheck) keep their byte-exact baseline; the live host and
/// the SkillCheck first-skill scenario opt in.&lt;/summary&gt;
public WorkerSim(bool withTutorial)
{
    _tutorial = withTutorial;
    _kpi = WorkerData.InitKpi;
    _mood = WorkerData.InitMood;
    _insp = WorkerData.InitInsp;
    HookReadyT = 0f; // HUD content is live from frame zero: no splash, no wall
}
/// &lt;summary&gt;First core op (Work/Slack toggle). No unlock, no cost, usable from t=0.
/// Op semantics owned here: one counted slack session per Work-&gt;Slack flip
/// (evt_04) and any op resets the idle timer (evt_05).&lt;/summary&gt;
public void SetWorking(bool working)
{
    if (working != _working &amp;&amp; !working) _slackSessions++;
```

```
    _working = working;
    _idleSeconds = 0f;
    if (FirstOpT &lt; 0f) FirstOpT = (float)_simTime;
}
// ----- T007: comedy skill tree face (doc section 3, 4 branches x 3 levels) -----
/// &lt;summary&gt;Learned levels of a branch (0 = not learned).&lt;/summary&gt;
public int SkillLevel(WorkerData.SkillBranch branch) =&gt; _skillLevels[(int)branch];
/// &lt;summary&gt;Next-level cost of a branch; -1 when the branch is maxed.&lt;/summary&gt;
public int SkillUpgradeCost(WorkerData.SkillBranch branch)
{
    int lv = _skillLevels[(int)branch];
    return lv &gt;= WorkerData.SkillLevelCount ? -1 : WorkerData.SkillUpgradeExpCost[lv];
}
/// &lt;summary&gt;Event-face exp reward channel (doc section 1: skill currency =
/// idle accrual + event rewards). Deterministic driver for tutorial and later
/// system slices (week/annual gifts land with T008).&lt;/summary&gt;
public void GrantExp(double amount)
{
    if (amount &lt;= 0.0) return;
    _exp = Math.Min(_exp + amount, WorkerData.ExpCap);
}
/// &lt;summary&gt;T009 offline-gain deposit channel: the offline claim banks
/// through the real wallet (never a parallel ledger), clamped at the coin
/// cap; non-positive amounts keep zero state change.&lt;/summary&gt;
public void DepositCoin(double amount)
{
    if (amount &lt;= 0.0) return;
    _coin = Math.Min(_coin + amount, WorkerData.CoinCap);
}
/// &lt;summary&gt;Sequential upgrade attempt: L0-&gt;L1-&gt;L2-&gt;L3 only (no skipping),
/// exact exp cost, refused with zero state change when broke or maxed.
/// A successful upgrade stamps the red-line first-skill beat, counts it and
/// drives the SYS04 skill-up hook (evt_11 slot, EventEngineCheck E11 law).&lt;/summary&gt;
public SkillUpResult TryUpgradeSkill(WorkerData.SkillBranch branch)
{
    int idx = (int)branch;
    int lv = _skillLevels[idx];
    if (lv &gt;= WorkerData.SkillLevelCount) return SkillUpResult.Maxed;
    int cost = WorkerData.SkillUpgradeExpCost[lv];
    if (_exp &lt; cost) return SkillUpResult.NoExp;
    _exp -= cost;
    _skillLevels[idx] = lv + 1;
    if (FirstSkillUpT &lt; 0f) FirstSkillUpT = (float)_simTime;
    SkillUpgradeCount++;
    NotifySkillUpgraded();
    return SkillUpResult.Ok;
}
/// &lt;summary&gt;Idempotent delta settle: continuous accrual, order-independent inputs.&lt;/summary&gt;
public void Tick(float dt)
{
```

```
if (dt &lt;= 0f) return;
// T013: both IAA windows burn on sim time in every branch - they are
// real-time offers, so they keep counting down inside the freeze show
bool boosting = _boostRemaining &gt; 0f;
if (boosting) _boostRemaining = Math.Max(0f, _boostRemaining - dt);
if (_skipCooldownRemaining &gt; 0f)
    _skipCooldownRemaining = Math.Max(0f, _skipCooldownRemaining - dt);
// T006-a forced-rest show (evt_07): progress frozen, time keeps flowing
if (_freezeRemaining &gt; 0f)
{
    _freezeRemaining -= dt;
    _simTime += dt;
    _idleSeconds += dt;
    return;
}
// wage main axis: base x rank coef x Ppt skill mult x NG+ inherit (four-factor chain)
// T013: the boost window rides the whole chain as a flat multiplier;
// with no boost the x1 multiply is byte-identical to the raw rate
float rate = WorkerData.WagePerSec(_rankIndex, _skillLevels[(int)WorkerData.SkillBranch.Ppt], _ngLoop)
           * (boosting ? WorkerData.BoostWageMult : 1f);
// T023 slice 1b wage-pause: the burnout strike show (mood below the strike
// line) and the KPI warn show pause the wage+exp accrual - ops stay free
// (a show, never a lockout), the cost is real coin. The beat keeps its
// per-second cadence but carries the honest amount, so the frozen counter
// IS the felt face (zero while paused).
bool wagesPaused = _strike || _warn;
if (!wagesPaused)
{
    _coin = Math.Min(_coin + (double)rate * dt, WorkerData.CoinCap);
    _exp = Math.Min(_exp + WorkerData.ExpPerSecOnline * dt, WorkerData.ExpCap);
}
// dual-axis deltas (doc section 2) + T008 flip shows: during the strike show
// the mood axis suspends its mode delta and only the natural +0.5/s regen
// applies, so every strike always lifts at 25 within ~10s of show time -
// the red law is "a show, never a lockout". The warn show keeps every rate
// untouched; only its own recovery rule (10s continuous Work) runs.
int report = _skillLevels[(int)WorkerData.SkillBranch.Report];
int slackArt = _skillLevels[(int)WorkerData.SkillBranch.SlackArt];
// T008-b2: the host hears the flip-show tuple only when it truly flips
bool showBefore = _strike;
bool warnBefore = _warn;
if (!_strike &amp;&amp; _mood &lt; WorkerData.MoodStrikeLine) { _strike = true; _strikeCount++; }
if (!_warn &amp;&amp; _kpi &lt; WorkerData.KpiWarnLine) { _warn = true; _warnCount++; _warnWorkSeconds = 0f;
float moodDelta = _strike ? WorkerData.MoodRecoverPerSec : WorkerData.MoodPerSec(_working, slackArt);
_kpi = ClampAxis(_kpi + WorkerData.KpiPerSec(_working, report, slackArt) * dt);
_mood = ClampAxis(_mood + moodDelta * dt);
_insp = ClampAxis(_insp + WorkerData.InspPerSec(_working) * dt);
if (_strike &amp;&amp; _mood &gt;= WorkerData.MoodRecoverTo) _strike = false;
if (_warn)
{
```

```
            if (_working)
            {
                _warnWorkSeconds += dt;
                if (_warnWorkSeconds &gt;= WorkerData.KpiRecoverWorkSeconds) _warn = false;
            }
            else _warnWorkSeconds = 0f;
        }
        if (_strike != showBefore || _warn != warnBefore) OnFlipShow?.Invoke(_strike, _warn);
        TryPromote();
        _simTime += dt;
        // T007 tutorial beat: the day-one red packet lands once inside the T+3-10s
        // window and funds exactly one L1 skill (event-face exp reward, doc section 3:
        // skill currency = idle accrual + event rewards).
        if (_tutorial &amp;&amp; !_tutorialGuideDone &amp;&amp; _simTime &gt;= TutorialGuideAtSec)
        {
            _tutorialGuideDone = true;
            _tutorialGuideT = (float)_simTime;
            _exp = Math.Min(_exp + TutorialRedPacketExp, WorkerData.ExpCap);
            OnTutorialGuide?.Invoke(_tutorialGuideT);
        }
        _wageBeatCarry += dt;
        while (_wageBeatCarry &gt;= WageBeatPeriodSec)
        {
            _wageBeatCarry -= WageBeatPeriodSec;
            EmitBeat(wagesPaused ? 0f : rate);
        }
        _idleSeconds += dt;
        TickFlow(dt);
    }
    /// &lt;summary&gt;T006-a: drive the SYS04 flow and answer every resolved card the
    /// same tick - fx land on the axes (clamped), freeze windows arm, the host
    /// hears the card through OnEventCard.&lt;/summary&gt;
    void TickFlow(float dt)
    {
        _flow.Tick(BuildFlowState(dt), _resolvedScratch);
        ApplyResolved(_resolvedScratch);
    }
    /// &lt;summary&gt;T006-b driver slice: the later system slices (week rollover,
    /// skill upgrade, annual settlement, promotion) translate their system beats
    /// into flow hook calls through these notify methods; the sim owns the
    /// fx/freeze/card application path so hook cards behave exactly like tick
    /// cards. T007 wires skill upgrades, T008 wires week/year/promotion.&lt;/summary&gt;
    public void NotifyWeekPassed() =&gt; DriveHook(_flow.OnWeekPassed);
    public void NotifySkillUpgraded() =&gt; DriveHook(_flow.OnSkillUpgraded);
    public void NotifyAnnualSettle() =&gt; DriveHook(_flow.OnAnnualSettle);
    public void NotifyPromotion(int newRankIndex)
    {
        _rankIndex = newRankIndex;
        DriveHook(_flow.OnStageGateMet);
    }
```

```
// ----- T008-a: promotion gates (doc section 4, dual track) + gate-4 dual ending -----
/// &lt;summary&gt;Gate-4 dual ending (doc section 4): the choice stays open at the CEO
/// rank. CeoRun keeps the endless run; Retire plays the financial-freedom ending
/// and rolls a NG+ loop: rank/axes/skills/exp reset, event flags cleared, wage
/// inherits x(1+0.10 per loop); coin stays - the career resets, the wallet does
/// not (share-card history keeps with T010). The ending slot itself resets to
/// None with the career, so the next run's CEO climb re-opens the choice
/// (multi-loop law: RetireCount/NG+ wage carry the history, not the slot).&lt;/summary&gt;
public enum EndingChoice { None = 0, CeoRun = 1, Retire = 2 }
/// &lt;summary&gt;Resolves the open ending; refused with zero state change when the
/// choice is not open (not at CEO yet, or already decided).&lt;/summary&gt;
public bool ChooseEnding(EndingChoice choice)
{
    if (_ending != EndingChoice.None) return false;
    if (_rankIndex &lt; (int)WorkerData.WorkerRank.Ceo || choice == EndingChoice.None) return false;
    _ending = choice;
    if (choice == EndingChoice.CeoRun) return true;
    _retireCount++;
    _ngLoop++;
    _rankIndex = 0;
    _kpi = WorkerData.InitKpi;
    _mood = WorkerData.InitMood;
    _insp = WorkerData.InitInsp;
    _exp = 0.0;
    Array.Clear(_skillLevels, 0, _skillLevels.Length);
    _gateMarkerMask = 0;
    _promoteOfferMask = 0;
    _strike = false;
    _warn = false;
    _warnWorkSeconds = 0f;
    _flow.Reset();
    _ending = EndingChoice.None;
    return true;
}
/// &lt;summary&gt;Promotion law (dual track): gates 0/1/3 promote only after their
/// marker card resolved (evt_04 slack rhythm / evt_02b weekly chain / evt_09
/// annual settle) AND the KPI threshold is met; gate 2's marker evt_12 is
/// stage-locked to the director rank by the frozen event table, so it IS the
/// ceremony card that fires through NotifyPromotion's OnStageGateMet drive at
/// the new rank - the KPI edge alone carries that gate. Whichever track lands
/// second fires the ceremony the same tick (promotion pacing: gate 1 inside P1).&lt;/summary&gt;
void TryPromote()
{
    int gate = _rankIndex;
    if (gate &gt;= WorkerData.GateKpiThreshold.Length) return;
    bool markerOk = gate == 2 || (_gateMarkerMask &amp; (1 &lt;&lt; gate)) != 0;
    bool kpiOk = _kpi &gt;= WorkerData.GateKpiThreshold[gate];
    // T010 slice 2: the funnel exposure row - the first moment either
    // promotion track lands at this gate (marker card armed or KPI over
    // the line) is the "offer" face of the promotion journey.
```

```
    if ((_promoteOfferMask &amp; (1 &lt;&lt; gate)) == 0 &amp;&amp; (markerOk || kpiOk))
    {
        _promoteOfferMask |= 1 &lt;&lt; gate;
        TrackPromote("offer");
    }
    if (!markerOk || !kpiOk) return;
    TrackPromote("ready");
    if (_firstPromoteT &lt; 0f) _firstPromoteT = (float)_simTime;
    _promoteCount++;
    NotifyPromotion(gate + 1);
    TrackPromote("promoted");
    OnPromoted?.Invoke(_rankIndex);
}
/// &lt;summary&gt;Gate marker bookkeeping: each frozen marker id (evt_04/evt_02b/
/// evt_12/evt_09) arms its gate bit the moment its card resolves.&lt;/summary&gt;
void TrackGateMarker(string id)
{
    int gate = Array.IndexOf(WorkerData.GateMarkerEventId, id);
    if (gate &gt;= 0) _gateMarkerMask |= 1 &lt;&lt; gate;
}
void DriveHook(Action&lt;WorkerEventFlow.FlowState, List&lt;WorkerEventFlow.ResolvedEvent&gt;&gt; hook)
{
    _resolvedScratch.Clear();
    hook(BuildFlowState(0f), _resolvedScratch);
    ApplyResolved(_resolvedScratch);
}
WorkerEventFlow.FlowState BuildFlowState(float dt)
{
    return new WorkerEventFlow.FlowState
    {
        Dt = dt,
        SimTime = (float)_simTime,
        Kpi = _kpi,
        Mood = _mood,
        Insp = _insp,
        RankIndex = _rankIndex,
        SlackSessions = _slackSessions,
        IdleSeconds = _idleSeconds
    };
}
void ApplyResolved(List&lt;WorkerEventFlow.ResolvedEvent&gt; resolved)
{
    for (int i = 0; i &lt; resolved.Count; i++)
    {
        var e = resolved[i];
        _kpi = ClampAxis(_kpi + e.Kpi);
        _mood = ClampAxis(_mood + e.Mood);
        _insp = ClampAxis(_insp + e.Insp);
        if (e.FreezeSeconds &gt; 0f) _freezeRemaining = e.FreezeSeconds;
        TrackGateMarker(e.Id);
```

```
            OnEventCard?.Invoke(e);
        }
    }
    void EmitBeat(float rate)
    {
        WageBeatCount++;
        var beat = new WageBeat
        {
            T = (float)_simTime,
            Amount = (double)rate * WageBeatPeriodSec,
            CoinAfter = _coin,
            Number = true,
            Pulse = true,
            Sound = true
        };
        if (FirstWageT &lt; 0f)
        {
            FirstWageT = beat.T;
            FirstWageThreeSync = beat.Number &amp;&amp; beat.Pulse &amp;&amp; beat.Sound &amp;&amp; beat.Amount &gt;
        }
        OnWageBeat?.Invoke(beat);
    }
    // ----- T012: save mirror restore face (red line 12: local authority) -----
    /// &lt;summary&gt;Restore the frozen save mirror into this sim, field-exact on
    /// the whole doc-section-10 face plus the two determinism carries, so a
    /// resumed run continues the same beat/op stream. Boundary: the event
    /// slate restarts fresh (flow internals are outside the mirror), the
    /// transient show/ceremony faces re-derive on their own - flip-shows
    /// re-arm from the axes on the next tick, gate markers re-arm through
    /// new event cards, the funnel and first-beat stamps restart with the
    /// resumed window. A null save (corrupt fallback) is a fresh-start
    /// no-op restore.&lt;/summary&gt;
    public void Restore(WorkerSaveData save)
    {
        if (save == null) return;
        _coin = save.coin;
        _exp = save.exp;
        _simTime = save.simTime;
        _wageBeatCarry = save.wageBeatCarry;
        _kpi = save.kpi;
        _mood = save.mood;
        _insp = save.insp;
        _working = save.working;
        _rankIndex = save.rankIndex;
        _ngLoop = save.ngLoop;
        _retireCount = save.retireCount;
        if (save.skillLevels != null)
            for (int i = 0; i &lt; _skillLevels.Length &amp;&amp; i &lt; save.skillLevels.Length; i++)
                _skillLevels[i] = save.skillLevels[i];
        _strike = false;
```

```
        _strikeCount = 0;
        _warn = false;
        _warnCount = 0;
        _warnWorkSeconds = 0f;
        _gateMarkerMask = 0;
        _promoteOfferMask = 0;
        _firstPromoteT = -1f;
        _promoteCount = 0;
        _ending = EndingChoice.None;
        _freezeRemaining = 0f;
        _idleSeconds = 0f;
        _slackSessions = 0;
        // T013: both IAA windows are transient session faces - they restart
        // empty on resume exactly like the show/ceremony faces above
        _skipCooldownRemaining = 0f;
        _boostRemaining = 0f;
        _flow.Reset();
    }
    /// &lt;summary&gt;Invariant round-trip fingerprint over the full live state: the dual-run byte-parity face.&lt;
    public string StateFingerprint()
    {
        var sb = new StringBuilder();
        sb.Append("v").Append(WorkerData.SchemaVersion).Append(';');
        sb.Append("t:").Append(_simTime.ToString("R", Inv)).Append(';');
        sb.Append("coin:").Append(_coin.ToString("R", Inv)).Append(';');
        sb.Append("exp:").Append(_exp.ToString("R", Inv)).Append(';');
        sb.Append("carry:").Append(_wageBeatCarry.ToString("R", Inv)).Append(';');
        sb.Append("axis:").Append(_kpi.ToString("R", Inv)).Append(',')
          .Append(_mood.ToString("R", Inv)).Append(',').Append(_insp.ToString("R", Inv)).Append(';');
        sb.Append("mode:").Append(_working ? "work" : "slack").Append(';');
        sb.Append("rank:").Append(_rankIndex).Append(",ng:").Append(_ngLoop).Append(';');
        sb.Append("skills:").Append(_skillLevels[0]).Append(',').Append(_skillLevels[1])
          .Append(',').Append(_skillLevels[2]).Append(',').Append(_skillLevels[3]).Append(';');
        sb.Append("r11:").Append(HookReadyT.ToString("R", Inv)).Append(',')
          .Append(FirstWageT.ToString("R", Inv)).Append(',').Append(FirstOpT.ToString("R", Inv))
          .Append(',').Append(WageBeatCount).Append(';');
        sb.Append("ops:").Append(_slackSessions).Append(',').Append(_idleSeconds.ToString("R", Inv)).Append(';');
        sb.Append("frz:").Append(_freezeRemaining.ToString("R", Inv)).Append(';');
        sb.Append("t8:").Append(_strike ? 1 : 0).Append(',').Append(_strikeCount).Append(',')
          .Append(_warn ? 1 : 0).Append(',').Append(_warnCount).Append(',')
          .Append(_warnWorkSeconds.ToString("R", Inv)).Append(',')
          .Append(_gateMarkerMask).Append(',')
          .Append(_firstPromoteT.ToString("R", Inv)).Append(',').Append(_promoteCount).Append(',')
          .Append((int)_ending).Append(',').Append(_retireCount).Append(';');
        sb.Append("tut:").Append(_tutorialGuideDone ? 1 : 0).Append(',')
          .Append(_tutorialGuideT.ToString("R", Inv)).Append(',')
          .Append(FirstSkillUpT.ToString("R", Inv)).Append(',').Append(SkillUpgradeCount).Append(';');
        sb.Append("iaa:").Append(_skipCooldownRemaining.ToString("R", Inv)).Append(',')
          .Append(_skipCount).Append(',')
          .Append(_boostRemaining.ToString("R", Inv)).Append(',').Append(_boostCount).Append(';');
```

```
            sb.Append("evt:").Append(_flow.EventSig());
            return sb.ToString();
        }
        public byte[] StateHash()
        {
            using (var md5 = MD5.Create())
                return md5.ComputeHash(Encoding.UTF8.GetBytes(StateFingerprint()));
        }
        static float ClampAxis(float v)
            =&gt; Math.Min(Math.Max(v, WorkerData.AxisMin), WorkerData.AxisMax);
        static readonly CultureInfo Inv = CultureInfo.InvariantCulture;
    }
}
// WorkerTexts.cs
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.G11.Game
{
    /// &lt;summary&gt;
    /// English-source text table (rule 8: English strings live in code, Chinese lives
    /// in the Configs/Texts_zhCN.json data file). Miss degrades to the key itself,
    /// matching the LocaleService contract. LookupWithZh is the ready-made delegate
    /// for LocaleService wiring: zh langs resolve from the loaded data table, other
    /// langs fall back to the English source.
    /// &lt;/summary&gt;
    public static class WorkerTexts
    {
        public const string LangZh = "zh-CN";
        static readonly (string key, string text)[] EnRows =
        {
            ("hook_boss_line", "Day one. Work hard, and I might let you breathe."),
            ("hook_slogan", "Idle office life: paychecks roll, ranks fly, jokes escalate."),
            // T019 polish: strongest-picture flash-forward copy (06-script rule 1,
            // the recording money shot): year-10 CEO penthouse tease + the snap-back
            // line that sells the grind back to day one.
            ("flash_ceo_line", "Flash-forward, year 10: your penthouse. Your name on the door."),
            ("flash_ceo_tag", "Snap back to day one. Grind on, intern."),
            ("rank_intern", "Intern"),
            ("rank_corporate", "Corporate Drone"),
            ("rank_teamlead", "Team Lead"),
            ("rank_director", "Director"),
            ("rank_ceo", "CEO"),
            ("hud_coin", "Wage"),
            ("hud_kpi", "KPI"),
            ("hud_mood", "Mood"),
            ("hud_insp", "Inspiration"),
            ("action_work", "Work"),
            ("action_slack", "Slack"),
            ("skill_ppt", "PPT Mastery"),
```

```
("skill_report", "The Art of Reporting"),
("skill_slackart", "Slackology"),
("skill_manageup", "Managing Up"),
("skill_ppt_l1", "Can Make Slides"),
("skill_ppt_l2", "Slides Have a Soul"),
("skill_ppt_l3", "One Slide Beats Ten"),
("skill_report_l1", "Explains Things"),
("skill_report_l2", "Boss Loves It"),
("skill_report_l3", "Reporting Is Art"),
("skill_slackart_l1", "Looks Busy"),
("skill_slackart_l2", "Leaves No Trace"),
("skill_slackart_l3", "Certified Slack Master"),
("skill_manageup_l1", "Knows How to Complain"),
("skill_manageup_l2", "Resources Lean Your Way"),
("skill_manageup_l3", "Boss's Inner Circle"),
("warn_mood_strike", "Mood bottomed out: your soul is on strike. Even the succulent is worried."),
("warn_mood_recover", "The soul returns to the desk. Mood recovering."),
("warn_kpi_letter", "KPI critical! HR is strolling over with a red card that is definitely not a bonus."),
("warn_kpi_recover", "Ten seconds of honest work later, the red card is quietly withdrawn."),
("promote_toast", "Promoted! New badge, new office, same absurdity."),
("ending_ceo", "CEO: the endless corner-office run."),
("ending_retire", "Financial freedom: retired with the office succulent."),
("ending_choice_title", "Top-Floor Crossroads: Run On or Retire?"),
("ending_ceo_toast", "The corner office is yours. The endless run continues."),
("ending_retire_toast", "Financial freedom! The wallet keeps its coins, the career starts over."),
("offline_summary_header", "While you were slacking, the paycheck kept coming:"),
("offline_summary_line", "Away {0} min, paid {1} coins, missed {2} absurd events."),
("offline_collect_btn", "Collect Paycheck"),
("offline_double_btn", "Watch Ad: Paycheck x2"),
("offline_collected_toast", "Payday: +{0} coins banked. The succulent approves."),
("offline_doubled_toast", "Overtime magic: +{0} coins. The ad gods smile."),
("offline_ad_fail_toast", "Ad not ready yet. The double offer stays - try again."),
("offline_capped_toast", "Today's double budget is spent. The plain payday still pays."),
("evt_01_name", "Pie-in-the-Sky Meeting"),
("evt_01_line", "Boss unveils a glowing pie chart: IPO next year. Trust the pie."),
("evt_01b_name", "The Pie Came True"),
("evt_01b_line", "Against all odds, the pie chart delivered. KPI rain!"),
("evt_02_name", "Weekly Report Literature Awards"),
("evt_02_line", "Your weekly report wins Best Fiction. It was 40 percent vibes."),
("evt_02b_name", "Praised by the Boss"),
("evt_02b_line", "The boss praises your report. The AI wrote it. Nobody asks."),
("evt_03_name", "Team-Building Social Death"),
("evt_03_line", "Karaoke. In front of everyone. Your manager picked a power ballad."),
("evt_04_name", "Paid Bathroom Break"),
("evt_04_line", "Five slack sessions earned you a throne of tranquility. Motivation restored."),
("evt_05_name", "The Succulent Wilted"),
("evt_05_line", "Your desk succulent has given up, exactly like your Monday self."),
("evt_05b_name", "The Succulent Lives"),
("evt_05b_line", "One drop of water and it is back. Some things are stronger than KPIs."),
("evt_06_name", "Boss Walks By"),
```

```
("evt_06_line", "Everyone switches screens at once. A perfectly synchronized ballet."),
("evt_07_name", "Overwork Hospital"),
("evt_07_line", "The drip is labeled deadline. The nurse says rest 30 seconds. Listen to the nurse."),
("evt_08_name", "The Optimization Letter"),
("evt_08_line", "HR sweetly hands you an envelope. Inside: a desk-relocation notice. Classic."),
("evt_09_name", "Year-End Riddle"),
("evt_09_line", "The bonus envelope holds a riddle. It pays {0} times your daily wage."),
("evt_10_name", "Pantry Gossip"),
("evt_10_line", "Tea-room intel unlocks inspiration. Someone microwaved fish. Again."),
("evt_11_name", "No Meeting Room"),
("evt_11_line", "Every room booked. The meeting is now a hallway stand-up. Very agile."),
("evt_12_name", "PUA Counter-Attack"),
("evt_12_line", "You reply: noted, and also no. The director blinks first. Legend status."),
("ad_double_offer", "Double this paycheck? One ad, zero dignity lost."),
("ad_gacha_pull", "Ten-pull the costume closet. Fashion is a workplace weapon."),
("skill_panel_title", "Skill Tree"),
("skill_panel_btn", "Skills"),
("skill_cost_fmt", "Next level: {0} exp"),
("skill_maxed_line", "MAX. The craft is complete."),
("skill_noexp_line", "Need {0} more exp. The grind continues."),
("skill_effect_wage", "Wage x{0}"),
("skill_effect_kpi", "KPI gain x{0}"),
("skill_effect_mood", "Slack mood x{0}"),
("skill_effect_mood_l3", "Slack mood x{0}, KPI loss halved"),
("skill_effect_offline", "Offline gain x{0}"),
("skill_up_toast", "{0} unlocked: {1}!"),
("tutorial_toast_title", "Day-One Red Packet"),
("tutorial_guide_line", "The boss stuffs a red packet in your hand: +{0} exp. Learn your first skill now."),
("gacha_panel_btn", "Outfits"),
("gacha_panel_title", "Costume Closet"),
("gacha_pull_btn", "Watch Ad: Ten-Pull"),
("gacha_empty_hint", "The closet is empty. One ad = ten pulls, one Epic guaranteed."),
("gacha_pity_line", "House rule: every ten-pull hides at least one Epic. Jackpot is a labor law."),
("gacha_stats_fmt", "Lifetime pulls {0} · Epics {1} · {2} pulls since last Epic"),
("offline_share_btn", "Brag Card"),
("offline_share_toast", "Brag card ready - go make the group chat jealous."),
("offline_share_fail_toast", "The card printer jammed. Nothing lost - try again."),
("gacha_granted_toast", "Ten fresh looks drop. The closet approves."),
("gacha_ad_fail_toast", "Ad not ready yet. The ten-pull stays - try again."),
("gacha_capped_toast", "Today's ten-pull budget is spent. Fashion resumes tomorrow."),
("skip_card_btn", "Skip This Meeting (Ad)"),
("skip_granted_toast", "Meeting skipped. Back to pretending to work."),
("skip_ad_fail_toast", "Ad not ready. The meeting drags on - try again."),
("skip_capped_toast", "Today's skip budget is spent. The show must go on."),
("skip_window_toast", "Skip is recharging. Even escape has a cooldown."),
("boost_btn", "2x Wage 30min (Ad)"),
("boost_active_fmt", "2x Wage ON - {0} min left"),
("boost_granted_toast", "Overtime approved: wages x2 for 30 minutes."),
("boost_ad_fail_toast", "Ad not ready. The raise waits - try again."),
("boost_capped_toast", "Today's boost budget is spent. Grind resumes tomorrow."),
```

```
("boost_window_toast", "Boost already running. Let it cook."),
("share_offline_title", "Slacked {0} min. The office paid anyway."),
("share_offline_line_coins", "+{0} coins banked while nobody watched"),
("share_offline_line_events", "{0} absurd meetings happened without you"),
("share_offline_line_hook", "Clock in. Slack off. Get promoted."),
("skin_wrinkled_shirt", "The Wrinkled Shirt"),
("skin_coffee_stain_tie", "The Coffee-Stain Tie"),
("skin_free_conf_shirt", "Free Conference Tee"),
("badge_plastic_lanyard", "Plastic Lanyard"),
("badge_carpool_sticker", "Carpool Club Sticker"),
("skin_short_sleeve_suit", "Short-Sleeve Suit"),
("skin_gym_bro_fit", "Gym-Bro Fit"),
("skin_silent_spec_auth", "Silent Spec Author"),
("badge_silver_lanyard", "Silver Lanyard"),
("skin_ceo_golden_suit", "CEO Golden Suit"),
("badge_penthouse_keycard", "Penthouse Keycard"),
("quest_panel_btn", "Quests"),
("quest_panel_title", "Daily Grind &amp;amp; Badge Wall"),
("quest_kind_slacks", "Touch the fish: {0}/{1} slack flips"),
("quest_kind_events", "Survive the meetings: {0}/{1} event cards"),
("quest_kind_skills", "Learn on the job: {0}/{1} skill-ups"),
("quest_kind_wages", "Collect paychecks: {0}/{1} wage beats"),
("quest_kind_promos", "Climb the ladder: {0}/{1} promotion"),
("quest_reward_fmt", "Reward: +{0} coins, +{1} exp"),
("quest_claimed_line", "Claimed - +{0} coins, +{1} exp banked."),
("quest_claim_toast", "Quest cashed: +{0} coins. The grind approves."),
("quest_progress_toast", "Still grinding: {0}/{1}."),
("badge_section_title", "Badge Wall"),
("badge_unlock_toast", "Badge unlocked: {0}!"),
("badge_locked_desc", "Keep grinding to earn this one."),
("badge_first_wage_name", "First Paycheck"),
("badge_first_wage_desc", "The first coin always hits hardest."),
("badge_pie_survivor_name", "Pie Meeting Survivor"),
("badge_pie_survivor_desc", "Sat through the glowing pie chart. Still kind of believes it."),
("badge_first_skill_name", "Crafts a Skill"),
("badge_first_skill_desc", "Learned something on company time. HR calls it growth."),
("badge_ladder_climber_name", "Ladder Climber"),
("badge_ladder_climber_desc", "One rung up. Same desk energy, better badge."),
("badge_toilet_master_name", "Paid Toilet Master"),
("badge_toilet_master_desc", "25 paid bathroom breaks. The throne room is yours."),
("badge_slack_grandmaster_name", "Slack Grandmaster"),
("badge_slack_grandmaster_desc", "100 slack flips and nobody noticed a thing."),
("badge_meeting_marathoner_name", "Meeting Marathoner"),
("badge_meeting_marathoner_desc", "50 absurd meetings survived. This could have been an email."),
("badge_paycheck_collector_name", "Paycheck Collector"),
("badge_paycheck_collector_desc", "600 wage beats banked. The succulent is proud."),
("badge_epic_wardrobe_name", "Epic Wardrobe"),
("badge_epic_wardrobe_desc", "Pulled an Epic fit. The office catwalk awaits."),
("badge_financial_freedom_name", "Financial Freedom"),
("badge_financial_freedom_desc", "Retired once. The wallet remembers."),
```

```
    // T020 compliance face (about/disclosure panel): the seven about
    // keys; zh twins live in Configs/Texts_zhCN.json (exact-parity law).
    ("about_button", "About"),
    ("about_title", "About This Game"),
    ("about_aigc_line", "Art and audio assets in this game are AI-assisted."),
    ("about_odds_line", "Costume ten-pull odds: each pull is {0}% common, {1}% rare, {2}% epic; if the first nin
    ("about_adcap_line", "Daily ad viewing is capped: outfit ten-pulls {0}, event skips {1}, boosts {2}, offline
    ("about_age_line", "Age rating: this work is suitable for ages 8 and up."),
    ("about_close", "Close")
};
static readonly Dictionary&lt;string, string&gt; En = BuildDict(EnRows);
static Dictionary&lt;string, string&gt; Zh;
static Dictionary&lt;string, string&gt; BuildDict((string key, string text)[] rows)
{
    var d = new Dictionary&lt;string, string&gt;(StringComparer.Ordinal);
    for (int i = 0; i &lt; rows.Length; i++) d[rows[i].key] = rows[i].text;
    return d;
}
/// &lt;summary&gt;Total English rows (dup keys collapse in the dict, so check compares both).&lt;/summary&gt;
public static int EnRowCount =&gt; EnRows.Length;
/// &lt;summary&gt;Unique English keys after dict collapse.&lt;/summary&gt;
public static int EnUniqueKeyCount =&gt; En.Count;
public static System.Collections.Generic.IReadOnlyList&lt;string&gt; EnKeys
{
    get
    {
        var keys = new List&lt;string&gt;(EnRows.Length);
        for (int i = 0; i &lt; EnRows.Length; i++) keys.Add(EnRows[i].key);
        return keys;
    }
}
/// &lt;summary&gt;English source read; miss degrades to the key (LocaleService contract).&lt;/summary&gt;
public static string GetEn(string key) =&gt; En.TryGetValue(key, out var t) &amp;&amp; !string.IsNullOrEmpty(t)
/// &lt;summary&gt;Parse the flat zhCN data table (Configs/Texts_zhCN.json) into the lookup.&lt;/summary&gt;
public static void LoadZhTable(string flatJson)
{
    var parsed = ParseFlatJson(flatJson);
    lock (Gate)
    {
        Zh = parsed;
    }
}
/// &lt;summary&gt;Count of loaded zh keys (0 when the data table was never loaded).&lt;/summary&gt;
public static int ZhKeyCount { get { lock (Gate) { return Zh != null ? Zh.Count : 0; } } }
/// &lt;summary&gt;LocaleService lookup delegate: zh langs read the data table, others read English.&lt;/summary
public static string LookupWithZh(string lang, string key)
{
    bool zh = !string.IsNullOrEmpty(lang) &amp;&amp; lang.IndexOf("zh", StringComparison.OrdinalIgnoreCase) &gt;
    if (zh)
    {
```

```
                lock (Gate)
                {
                    if (Zh != null &amp;&amp; Zh.TryGetValue(key, out var t) &amp;&amp; !string.IsNullOrEmpty(t)) return
                }
            }
            return GetEn(key);
        }
        static readonly object Gate = new object();
        /// &lt;summary&gt;Minimal flat-JSON parser: {"k":"v","k2":"v2"} with basic escapes, no nesting.&lt;/summary&gt;
        static Dictionary&lt;string, string&gt; ParseFlatJson(string json)
        {
            var result = new Dictionary&lt;string, string&gt;(StringComparer.Ordinal);
            if (string.IsNullOrEmpty(json)) return result;
            int i = 0;
            int n = json.Length;
            while (i &lt; n)
            {
                int keyStart = json.IndexOf('"', i);
                if (keyStart &lt; 0) break;
                int keyEnd = json.IndexOf('"', keyStart + 1);
                if (keyEnd &lt; 0) break;
                int colon = json.IndexOf(':', keyEnd + 1);
                if (colon &lt; 0) break;
                int valStart = json.IndexOf('"', colon + 1);
                if (valStart &lt; 0) break;
                int valEnd = valStart + 1;
                var val = new StringBuilder();
                while (valEnd &lt; n)
                {
                    char c = json[valEnd];
                    if (c == '\\')
                    {
                        char next = valEnd + 1 &lt; n ? json[valEnd + 1] : '\0';
                        if (next == 'n') val.Append('\n');
                        else if (next == 't') val.Append('\t');
                        else val.Append(next);
                        valEnd += 2;
                        continue;
                    }
                    if (c == '"') break;
                    val.Append(c);
                    valEnd++;
                }
                result[json.Substring(keyStart + 1, keyEnd - keyStart - 1)] = val.ToString();
                i = valEnd + 1;
            }
            return result;
        }
    }
}
```